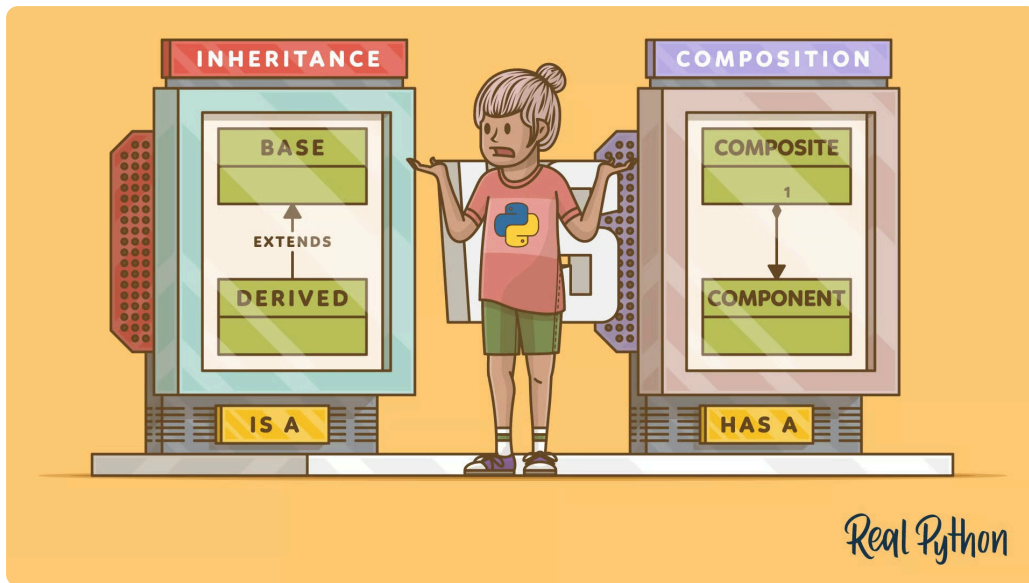




Inheritance and Composition: A Python OOP Guide

by Isaac Rodriguez 📅 Jan 11, 2025 📖 1h 19m 💬 28 Comments

🔑 intermediate 📌 best-practices 🐍 python

Mark as Completed



🔗 Share

Table of Contents

- What Are Inheritance and Composition?
 - What's Inheritance?
 - What's Composition?
- An Overview of Inheritance in Python
 - The Object Super Class
 - Exceptions Are an Exception
 - Creating Class Hierarchies
 - Abstract Base Classes in Python
 - Implementation Inheritance vs Interface Inheritance
 - The Class Explosion Problem
 - Inheriting Multiple Classes
- Composition in Python
 - Flexible Designs With Composition
 - Customizing Behavior With Composition



- [Mixing Features With Mixin Classes](#)
- [Composition to Model “Has A” Relationship](#)
- [Composition to Change Run-Time Behavior](#)
- [Choosing Between Inheritance and Composition in Python](#)
- [Conclusion](#)
- [Recommended Reading](#)
- [Frequently Asked Questions](#)

 [Remove ads](#)

Recommended Course

[Inheritance and Composition: A Python OOP Guide](#) (3h 34m)

In Python, understanding inheritance and composition is crucial for effective object-oriented programming. Inheritance allows you to model an **is a** relationship, where a derived class extends the functionality of a base class. Composition, on the other hand, models a **has a** relationship, where a class contains objects of other classes to build complex structures. Both techniques promote code reuse, but they approach it differently.

By the end of this tutorial, you’ll understand that:

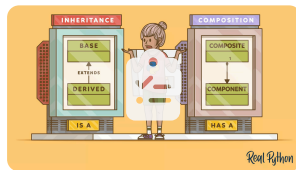
- **Composition and inheritance** in Python model relationships between classes, enabling code reuse in different ways.
- **Composition** is achieved by creating classes that contain objects of other classes, allowing for flexible designs.
- **Inheritance** models an **is a** relationship, allowing derived classes to extend base class functionality.
- **Inheritance** in Python is achieved by defining classes that derive from base classes, inheriting their interface and implementation.

Exploring the differences between inheritance and composition helps you choose the right approach for designing robust, maintainable Python applications. Understanding how and when to



Get Your Code: Click here to get the free sample code that shows you how to use inheritance and composition in Python.

 **Take the Quiz:** Test your knowledge with our interactive “Inheritance and Composition: A Python OOP Guide” quiz. You’ll receive a score upon completion to help you track your learning progress:



Interactive Quiz

Inheritance and Composition: A Python OOP Guide

In this quiz, you'll test your understanding of inheritance and composition in Python. These are two major concepts in object-oriented programming that help model the relationship between two classes. By working through this quiz, you'll revisit how to use inheritance and composition in Python, model class hierarchies, and use multiple inheritance.

What Are Inheritance and Composition?

Inheritance and **composition** are two major concepts in object-oriented programming that model the relationship between two classes. They drive the design of an application and determine how the application should evolve as new features are added or requirements change.

Both of them enable code reuse, but they do it in different ways.

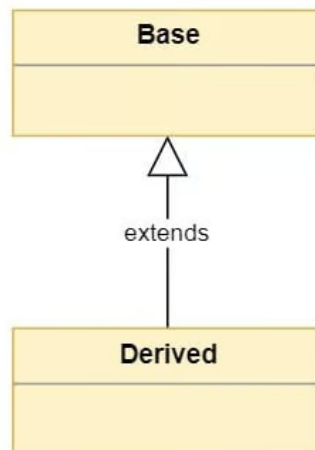
 Remove ads

What's Inheritance?



you've created a relationship where `Derived` **is a** specialized version of `Base`.

Inheritance is represented using the [Unified Modeling Language](#), or UML, in the following way:



This model represents classes as boxes with the class name on top. It represents the inheritance relationship with an arrow from the derived class pointing to the base class. The word **extends** is usually added to the arrow.

Note: In an inheritance relationship:

- Classes that inherit from another are called derived classes, subclasses, or subtypes.
- Classes from which other classes are derived are called base classes or super classes.
- A derived class is said to derive, inherit, or extend a base class.

Say you have the base class `Animal`, and you derive from it to create a `Horse` class. The inheritance relationship states that `Horse` **is an** `Animal`. This means that `Horse` inherits the [interface](#) and implementation of `Animal`, and you can use `Horse` objects to replace `Animal` objects in the application.



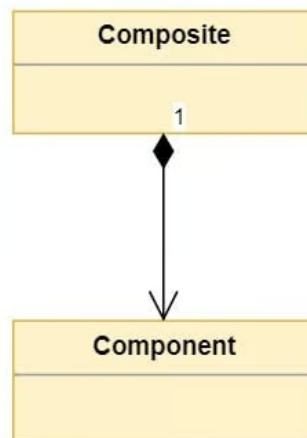
objects of type `S` doesn't change the program's behavior.

You'll see in this tutorial why you should always follow the Liskov substitution principle when creating your class hierarchies, and you'll learn about the problems that you'll run into if you don't.

What's Composition?

Composition is a concept that models a **has a** relationship. It enables creating complex types by combining objects of other types. This means that a class `Composite` can contain an object of another class `Component`. This relationship means that a `Composite` **has a** `Component`.

UML represents composition as follows:



The model represents composition through a line that starts with a diamond at the composite class and points to the component class. The composite side can express the cardinality of the relationship. The **cardinality** indicates the number or the valid range of `Component` instances that the `Composite` class will contain.

In the diagram above, the 1 represents that the `Composite` class contains one object of type `Component`. You can express cardinality in the following ways:

- **A number** indicates the number of `Component` instances that `Composite` contains.



- A **range 1..4** indicates that the `Composite` class can contain a range of `Component` instances. You indicate the range with the minimum and maximum number of instances, or minimum and many instances like in **1..***.

Note: Classes that contain objects of other classes are usually referred to as composites, while classes that are used to create more complex types are referred to as components.

For example, your `Horse` class can be composed by another object of type `Tail`. Composition allows you to express that relationship by saying `Horse` **has a** `Tail`.

Composition enables you to reuse code by adding objects to other objects, as opposed to inheriting the interface and implementation of other classes. Both `Horse` and `Dog` classes can leverage the functionality of `Tail` through composition without deriving one class from the other.

An Overview of Inheritance in Python

Everything in Python is an object. Modules are objects, class definitions and functions are objects, and of course, objects created from classes are objects too.

Inheritance is a required feature of every object-oriented programming language. This means that Python supports inheritance, and as you'll see later, it's one of the few languages that supports multiple inheritance.

When you write Python code using classes, you're using inheritance even if you don't know that you're using it. Next up, take a look at what that means.

 Remove ads



Python interactive shell and write a little bit of code. You'll start by writing the simplest class possible:

Python



```
>>> class EmptyClass:
...     pass
... 
```

You declared `EmptyClass`, which doesn't do much, but it'll illustrate the most basic inheritance concepts. Now that you have the class declared, you can create an instance of the class and use the `dir()` function to list its members:

Python



```
>>> c = EmptyClass()
>>> dir(c)
['__class__', '__delattr__', '__dict__', '__dir__', '__doc__',
 '__format__', '__ge__', '__getattr__', '__getstate__',
 '__hash__', '__init__', '__init_subclass__', '__le__', '__module__',
 '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
 '__setattr__', '__sizeof__', '__str__', '__subclasshook__']
```

The `dir()` function returns a list of all the members in the specified object. You haven't declared any members in `EmptyClass`, so where's the list coming from? You can find out using the interactive interpreter:

Python



```
>>> o = object()
>>> dir(o)
['__class__', '__delattr__', '__dir__', '__doc__', '__eq__',
 '__format__', '__ge__', '__getattr__', '__getstate__',
 '__hash__', '__init__', '__init_subclass__', '__le__', '__module__',
 '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__',
 '__setattr__', '__sizeof__', '__str__', '__subclasshook__']
```

As you can see, the two lists are nearly identical. There are three additional members in `EmptyClass`:



3. `__weakref__`

However, every single member of the object class is also present in `EmptyClass`.

This is because every class that you create in Python implicitly derives from `object`. You could be more explicit and write `class EmptyClass(object):`, but it's redundant and unnecessary.

Note: In Python 2, you had to explicitly derive from `object` for reasons beyond the scope of this tutorial, but you can read about it in the [new-style and classic classes](#) section of the [Python 2 documentation](#).

Okay, it's not entirely true that every class in Python derives from `object`. There's one aptly named exception, which you'll learn about next.

Exceptions Are an Exception

Every class that you create in Python will implicitly derive from `object`. However, there's one exception to this rule: classes used to indicate errors by raising an [exception](#).

If you try to treat a normal Python class like an exception and `raise` it, then Python will present you with a `TypeError`:

Python



```
>>> class NotAnError:
...     pass
...
>>> raise NotAnError()
Traceback (most recent call last):
...
TypeError: exceptions must derive from BaseException
```



exception, but the output states that the exception is of type `TypeError`, not `NotAnError`, and that all exceptions must derive from `BaseException`.

`BaseException` is a base class provided for all error types. To create a new error type, you must derive your class from `BaseException` or one of its derived classes. The convention in Python is to derive your custom error types from `Exception`, which in turn derives from `BaseException`.

The correct way to define your error type is the following:

Python



```
>>> class AnError(Exception):
...     pass
...
>>> raise AnError()
Traceback (most recent call last):
...
AnError
```

In this example, `AnError` explicitly inherits from `Exception` instead of implicitly inheriting from `object`. With that change, you've fulfilled the requirements for creating a custom exception, and you can now raise your new exception class. When you raise `AnError`, the output correctly states that Python raised an error of the type `AnError`.

Remove ads

Creating Class Hierarchies

Inheritance is the mechanism that you'll use to create hierarchies of related classes. These related classes will share a common interface that the base classes will define. Derived classes can specialize the interface by providing a particular implementation where applicable.

In this section, you'll start modeling an HR system. Along the way, you'll explore the use of inheritance and see how derived classes can provide a concrete implementation of the base class interface.



now their payroll is calculated.

You start by implementing a `PayrollSystem` class that processes payroll:

```
Python                                                                    hr.py

class PayrollSystem:
    def calculate_payroll(self, employees):
        print("Calculating Payroll")
        print("=====")
        for employee in employees:
            print(f"Payroll for: {employee.id} - {employee.name}")
            print(f"- Check amount: {employee.calculate_payroll()}")
            print("")
```

`PayrollSystem` implements a `.calculate_payroll()` method that takes a collection of employees and prints their `.id`, `.name`, and check amount using the `.calculate_payroll()` method exposed on each employee object.

Now, you implement a base class, `Employee`, that handles the common interface for every employee type:

```
Python                                                                    hr.py

# ...

class Employee:
    def __init__(self, id, name):
        self.id = id
        self.name = name
```

`Employee` is the base class for all employee types. It's constructed with an `.id` and a `.name`. What you're saying is that every `Employee` must have an `.id` as well as a `.name` assigned.

The HR system requires that every `Employee` processed must provide a `.calculate_payroll()` interface that returns the weekly salary for the employee. The implementation of that interface differs depending on the type of `Employee`.



Python

hr.py

...

```
class SalaryEmployee(Employee):
    def __init__(self, id, name, weekly_salary):
        super().__init__(id, name)
        self.weekly_salary = weekly_salary

    def calculate_payroll(self):
        return self.weekly_salary
```

You create a derived class, `SalaryEmployee`, that inherits from `Employee`. The class initializes with the `.id` and `.name` required by the base class, and you use `super()` to initialize the members of the base class. You can read all about `super()` in [Supercharge Your Classes With Python `super\(\)`](#).

`SalaryEmployee` also requires a `weekly_salary` initialization parameter that represents the amount that the employee makes per week.

The class provides the required `.calculate_payroll()` method that the HR system uses. The implementation just returns the amount stored in `weekly_salary`.

The company also employs manufacturing workers who are paid by the hour, so you add `HourlyEmployee` to the HR system:

Python

hr.py

...

```
class HourlyEmployee(Employee):
    def __init__(self, id, name, hours_worked, hourly_rate):
        super().__init__(id, name)
        self.hours_worked = hours_worked
        self.hourly_rate = hourly_rate

    def calculate_payroll(self):
        return self.hours_worked * self.hourly_rate
```



calculate the payroll. You implement the `.calculate_payroll()` method by returning the hours worked times the hourly rate.

Finally, the company employs sales associates who are paid through a fixed salary plus a commission based on their sales, so you create a `CommissionEmployee` class:

```
Python                                     hr.py

# ...

class CommissionEmployee(SalaryEmployee):
    def __init__(self, id, name, weekly_salary, commission):
        super().__init__(id, name, weekly_salary)
        self.commission = commission

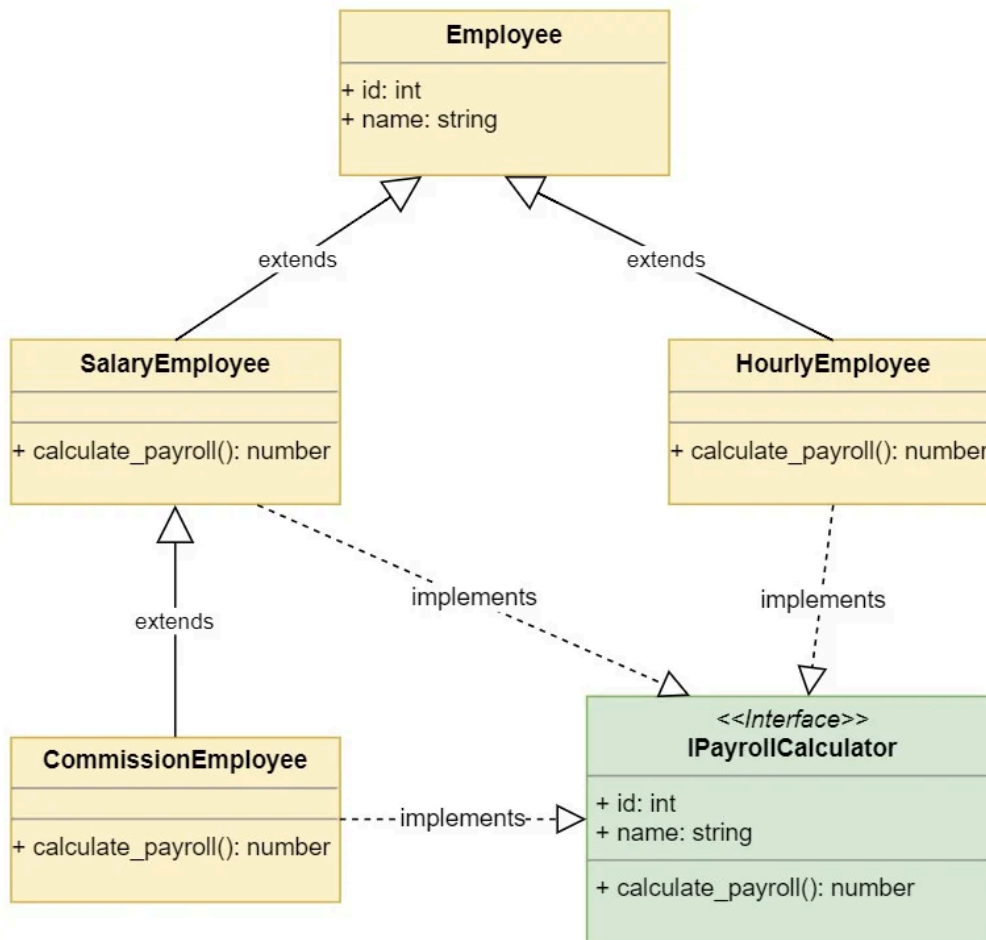
    def calculate_payroll(self):
        fixed = super().calculate_payroll()
        return fixed + self.commission
```

You derive `CommissionEmployee` from `SalaryEmployee` because both classes have a `weekly_salary` to consider. At the same time, you initialize `CommissionEmployee` with a `commission` value that's based on the sales for the employee.

With `.calculate_payroll()`, you leverage the implementation of the base class to retrieve the fixed salary, and you add the commission value.

Since `CommissionEmployee` derives from `SalaryEmployee`, you have access to the `weekly_salary` property directly, and you could've implemented `.calculate_payroll()` using the value of that property.

The problem with accessing the property directly is that if the implementation of `SalaryEmployee.calculate_payroll()` changes, then you'll have to also change the implementation of `CommissionEmployee.calculate_payroll()`. It's better to rely on the already-implemented method in the base class and extend the functionality as needed.



The diagram shows the inheritance hierarchy of the classes. The derived classes implement the `IPayrollCalculator` interface, which the `PayrollSystem` requires. The `PayrollSystem.calculate_payroll()` implementation requires that the objects in the `employees` collection contain an `.id`, `.name`, and `.calculate_payroll()` implementation.

Note: Interfaces are represented similarly to classes in UML diagrams, with the word *Interface* above the interface name. Interface names are usually prefixed with a capital `I`.

In Python, you don't implement interfaces explicitly. Instead, interfaces are defined by the attributes used and methods called by other functions and methods.



payroll:

```
Python                                program.py

import hr

salary_employee = hr.SalaryEmployee(1, "John Smith", 1500)
hourly_employee = hr.HourlyEmployee(2, "Jane Doe", 40, 15)
commission_employee = hr.CommissionEmployee(3, "Kevin Baco

payroll_system = hr.PayrollSystem()
payroll_system.calculate_payroll(
    [salary_employee, hourly_employee, commission_employee
    )
```

You can run the program in the command line and see the results:

Shell



```
$ python program.py
```

```
Calculating Payroll
=====
Payroll for: 1 - John Smith
- Check amount: 1500

Payroll for: 2 - Jane Doe
- Check amount: 600

Payroll for: 3 - Kevin Bacon
- Check amount: 1250
```

The program creates three employee objects, one for each of the derived classes. Then, it creates the payroll system and passes a list of the employees to its `.calculate_payroll()` method, which calculates the payroll for each employee and prints the results.

Notice how the `Employee` base class doesn't define a `.calculate_payroll()` method. This means that if you were to create a plain `Employee` object and pass it to the `PayrollSystem`, then you'd get an error. You can try it in the Python interactive interpreter:



```
Python                                     hr.py

from abc import ABC, abstractmethod

# ...

class Employee(ABC):
    def __init__(self, id, name):
        self.id = id
        self.name = name

    @abstractmethod
    def calculate_payroll(self):
        pass
```

You derive `Employee` from `ABC`, making it an abstract base class. Then, you decorate the `.calculate_payroll()` method with the `@abstractmethod` decorator.

This change has two nice side-effects:

1. You're telling users of the module that objects of type `Employee` can't be created.
2. You're telling other developers working on the `hr` module that if they derive from `Employee`, then they must override the `.calculate_payroll()` abstract method.

You can see that you can't create objects of type `Employee` anymore using the interactive interpreter:

```
Python                                     

>>> import hr
>>> employee = hr.Employee(1, "Abstract")
Traceback (most recent call last):
...
TypeError: Can't instantiate abstract class Employee
  with abstract method calculate_payroll
```

The output shows that you can't instantiate the class because it contains an abstract method, `.calculate_payroll()`. Derived



You can feel this rule directly. In the interactive example below, toggle which abstract methods the `Dog` subclass implements and watch how, just like `Employee`, the class can't be instantiated until every abstract method is implemented:

Implement in `Dog` : ☐ `speak()` ☐ `move()`

Reset

```
animals.py

from abc import ABC, abstractmethod

class Animal(ABC):
    @abstractmethod
    def speak(self):
        pass

    @abstractmethod
    def move(self):
        pass

    def jump(self):
        return f"{self.__class__.__name__} is jumping"

class Dog(Animal):
    pass
```

Python REPL

X `Dog()` raises `TypeError`

```
>>> from animals import Dog
>>> dog = Dog()
Traceback (most recent call last):
...
TypeError: Can't instantiate abstract class Dog without an
implementation for abstract methods 'move', 'speak'
```

Implementation Inheritance vs Interface Inheritance



1. **The base class interface:** The derived class inherits all the methods, properties, and attributes of the base class.
2. **The base class implementation:** The derived class inherits the code that implements the class interface.

Most of the time, you'll want to inherit the implementation of a class, but you'll want to implement multiple interfaces so that you can use your objects in different situations.

Modern programming languages are designed with this basic concept in mind. They allow you to inherit from a single class, but you can implement multiple interfaces.

In Python, you don't have to explicitly declare an interface. Any object that implements the desired interface can be used in place of another object. This is known as **duck typing**. Duck typing is usually explained as *if it walks like a duck and it quacks like a duck, then it must be a duck*. In other words, it's enough to behave like a duck to be considered a duck.

To illustrate this, you'll now add a `DisgruntledEmployee` class to the example above, and it won't derive from `Employee`. Create a new file called `disgruntled.py` and add the following code:

```
Python                                disgruntled.py

class DisgruntledEmployee:
    def __init__(self, id, name):
        self.id = id
        self.name = name

    def calculate_payroll(self):
        return 1_000_000
```

The `DisgruntledEmployee` class doesn't derive from `Employee`, but it exposes the same interface that `PayrollSystem` requires. Remember that `PayrollSystem.calculate_payroll()` requires a list of objects that implement the following interface:



name

- A `.calculate_payroll()` method that doesn't take any parameters and returns the payroll amount to process

The `DisgruntledEmployee` class meets all these requirements, so `PayrollSystem` can still calculate its payroll.

You can modify the program to use the `DisgruntledEmployee` class:

```
Python                                program.py

import hr
import disgruntled

salary_employee = hr.SalaryEmployee(1, "John Smith", 1500)
hourly_employee = hr.HourlyEmployee(2, "Jane Doe", 40, 15)
commission_employee = hr.CommissionEmployee(3, "Kevin Baco
disgruntled_employee = disgruntled.DisgruntledEmployee(200

payroll_system = hr.PayrollSystem()
payroll_system.calculate_payroll(
    [
        salary_employee,
        hourly_employee,
        commission_employee,
        disgruntled_employee,
    ]
)
```

The program creates a `DisgruntledEmployee` object and adds it to the list that `PayrollSystem` processes. You can now run the program and see its output:

Shell



```
$ python program.py
```

```
Calculating Payroll
```

```
=====
```

```
Payroll for: 1 - John Smith
- Check amount: 1500
```

```
Payroll for: 2 - Jane Doe
```



- Check amount: 1250

Payroll for: 20000 - Anonymous

- Check amount: 1000000

As you can see, the PayrollSystem can still process the new object because it meets the desired interface.

Since you don't have to derive from a specific class for your objects to be reusable by the program, you may be asking why you should use inheritance instead of just implementing the desired interface. The following rules may help you to make this decision:

- **Use inheritance to reuse an implementation:** Your derived classes should leverage most of their base class implementation. They must also model an **is a** relationship. A Customer class might also have an `.id` and a `.name`, but a Customer is not an Employee, so in this case, you shouldn't use inheritance.
- **Implement an interface to be reused:** When you want your class to be reused by a specific part of your application, you implement the required interface in your class, but you don't need to provide a base class, or inherit from another class.

You can now clean up the example above to move on to the next topic. You can delete the `disgruntled.py` file and then modify the `hr` module to its original state:

Python

hr.py

```
class PayrollSystem:
    def calculate_payroll(self, employees):
        print("Calculating Payroll")
        print("=====")
        for employee in employees:
            print(f"Payroll for: {employee.id} - {employee.name}")
            print(f"- Check amount: {employee.calculate_payroll()}")
            print("")

class Employee:
    def __init__(self, id, name):
```



```
class SalaryEmployee(Employee):
    def __init__(self, id, name, weekly_salary):
        super().__init__(id, name)
        self.weekly_salary = weekly_salary

    def calculate_payroll(self):
        return self.weekly_salary

class HourlyEmployee(Employee):
    def __init__(self, id, name, hours_worked, hourly_rate):
        super().__init__(id, name)
        self.hours_worked = hours_worked
        self.hourly_rate = hourly_rate

    def calculate_payroll(self):
        return self.hours_worked * self.hourly_rate

class CommissionEmployee(SalaryEmployee):
    def __init__(self, id, name, weekly_salary, commission):
        super().__init__(id, name, weekly_salary)
        self.commission = commission

    def calculate_payroll(self):
        fixed = super().calculate_payroll()
        return fixed + self.commission
```

You removed the import of the `abc` module since the `Employee` class doesn't need to be abstract. You also removed the abstract `.calculate_payroll()` method from it since it doesn't provide any implementation.

Basically, you're inheriting the implementation of the `.id` and `.name` attributes of the `Employee` class in your derived classes. Since `.calculate_payroll()` is just an interface to the `PayrollSystem.calculate_payroll()` method, you don't need to implement it in the `Employee` base class.

Notice how the `CommissionEmployee` class derives from `SalaryEmployee`. This means that `CommissionEmployee` inherits the implementation and interface of `SalaryEmployee`. You can see how the `CommissionEmployee.calculate_payroll()` method leverages



 Remove ads

The Class Explosion Problem

If you're not careful, inheritance can lead you to a huge hierarchical class structure that's hard to understand and maintain. This is known as the **class explosion problem**.

You started building a class hierarchy of `Employee` types used by the `PayrollSystem` to calculate payroll. Now, you need to add some functionality to those classes so that you can use them with the new `ProductivitySystem`.

`ProductivitySystem` tracks productivity based on employee roles. There are different employee roles:

- **Managers:** They walk around yelling at people, telling them what to do. They're salaried employees and make more money.
- **Secretaries:** They do all the paperwork for managers and ensure that everything gets billed and payed on time. They're also salaried employees but make less money.
- **Sales employees:** They make a lot of phone calls to sell products. They have a salary, but they also get commissions for sales.
- **Factory workers:** They manufacture the products for the company. They're paid by the hour.

With those requirements, you start to see that `Employee` and its derived classes might belong somewhere other than the `hr` module because now they're also used by the `ProductivitySystem`.

You create an `employees` module and move the classes there:

```
Python                                     employees.py

class Employee:
    def __init__(self, id, name):
        self.id = id
```



```
def __init__(self, id, name, weekly_salary):
    super().__init__(id, name)
    self.weekly_salary = weekly_salary

def calculate_payroll(self):
    return self.weekly_salary

class HourlyEmployee(Employee):
    def __init__(self, id, name, hours_worked, hourly_rate):
        super().__init__(id, name)
        self.hours_worked = hours_worked
        self.hourly_rate = hourly_rate

    def calculate_payroll(self):
        return self.hours_worked * self.hourly_rate

class CommissionEmployee(SalaryEmployee):
    def __init__(self, id, name, weekly_salary, commission):
        super().__init__(id, name, weekly_salary)
        self.commission = commission

    def calculate_payroll(self):
        fixed = super().calculate_payroll()
        return fixed + self.commission
```

The implementation remains the same, but you move the classes to the `employees` module. Your `hr` module is now much smaller and focused on the payroll system.

Python

`hr.py`

```
class PayrollSystem:
    def calculate_payroll(self, employees):
        print("Calculating Payroll")
        print("=====")
        for employee in employees:
            print(f"Payroll for: {employee.id} - {employee.name}")
            print(f"- Check amount: {employee.calculate_payroll()}")
            print("")
```

With both `hr.py` and `employees.py` in place, you can now update your program to support the change:



```
import employees

salary_employee = employees.SalaryEmployee(1, "John Smith")
hourly_employee = employees.HourlyEmployee(2, "Jane Doe", .
commission_employee = employees.CommissionEmployee(3, "Kev

payroll_system = hr.PayrollSystem()
payroll_system.calculate_payroll(
    [salary_employee, hourly_employee, commission_employee
    )
```

You run the program and verify that it still works:

Shell



```
$ python program.py
```

```
Calculating Payroll
```

```
=====
```

```
Payroll for: 1 - John Smith
```

```
- Check amount: 1500
```

```
Payroll for: 2 - Jane Doe
```

```
- Check amount: 600
```

```
Payroll for: 3 - Kevin Bacon
```

```
- Check amount: 1250
```

With everything in place, you start adding the new classes:

Python

employees.py

```
# ...
```

```
class Manager(SalaryEmployee):
```

```
    def work(self, hours):
```

```
        print(f"{self.name} screams and yells for {hours}")
```

```
class Secretary(SalaryEmployee):
```

```
    def work(self, hours):
```

```
        print(f"{self.name} expends {hours} hours doing of
```

```
class SalesPerson(CommissionEmployee):
```

```
    def work(self, hours):
```



```
def work(self, hours):
    print(f"{self.name} manufactures gadgets for {hour
```

First, you add a `Manager` class that derives from `SalaryEmployee`. The class exposes a `.work()` method that the productivity system will use. The method takes the `hours` that the employee worked.

Then you add `Secretary`, `SalesPerson`, and `FactoryWorker` and then implement the `.work()` interface, so they can be used by the productivity system—which you haven't created yet.

As a next step, you can create a new file called `productivity.py` and add the `ProductivitySystem` class:

```
Python                                productivity.py

class ProductivitySystem:
    def track(self, employees, hours):
        print("Tracking Employee Productivity")
        print("=====")
        for employee in employees:
            employee.work(hours)
        print("")
```

The class tracks employees in the `.track()` method that takes a list of employees and the number of hours to track. As outlined above, the productivity system makes use of `.work()` on each of the objects in `employees` to accomplish the tracking.

You can now add the productivity system to your program, and update it to represent different types of employees:

```
Python                                program.py

import hr
import employees
import productivity

manager = employees.Manager(1, "Mary Poppins", 3000)
secretary = employees.Secretary(2, "John Smith", 1500)
sales_guy = employees.SalesPerson(3, "Kevin Bacon", 1000, ...
```



```

        secretary,
        sales_guy,
        factory_worker,
    ]

    productivity_system = productivity.ProductivitySystem()
    productivity_system.track(employees, 40)

    payroll_system = hr.PayrollSystem()
    payroll_system.calculate_payroll(employees)

```

Your updated program creates a list of employees of different types. The employee list is sent to the productivity system to track their work for forty hours. Then the same list of employees is sent to the payroll system to calculate their payroll.

You can run the program to see the output:

Shell



```
$ python program.py
```

```

Tracking Employee Productivity
=====
Mary Poppins screams and yells for 40 hours.
John Smith expends 40 hours doing office paperwork.
Kevin Bacon expends 40 hours on the phone.
Jane Doe manufactures gadgets for 40 hours.

Calculating Payroll
=====
Payroll for: 1 - Mary Poppins
- Check amount: 3000

Payroll for: 2 - John Smith
- Check amount: 1500

Payroll for: 3 - Kevin Bacon
- Check amount: 1250

Payroll for: 4 - Jane Doe
- Check amount: 600

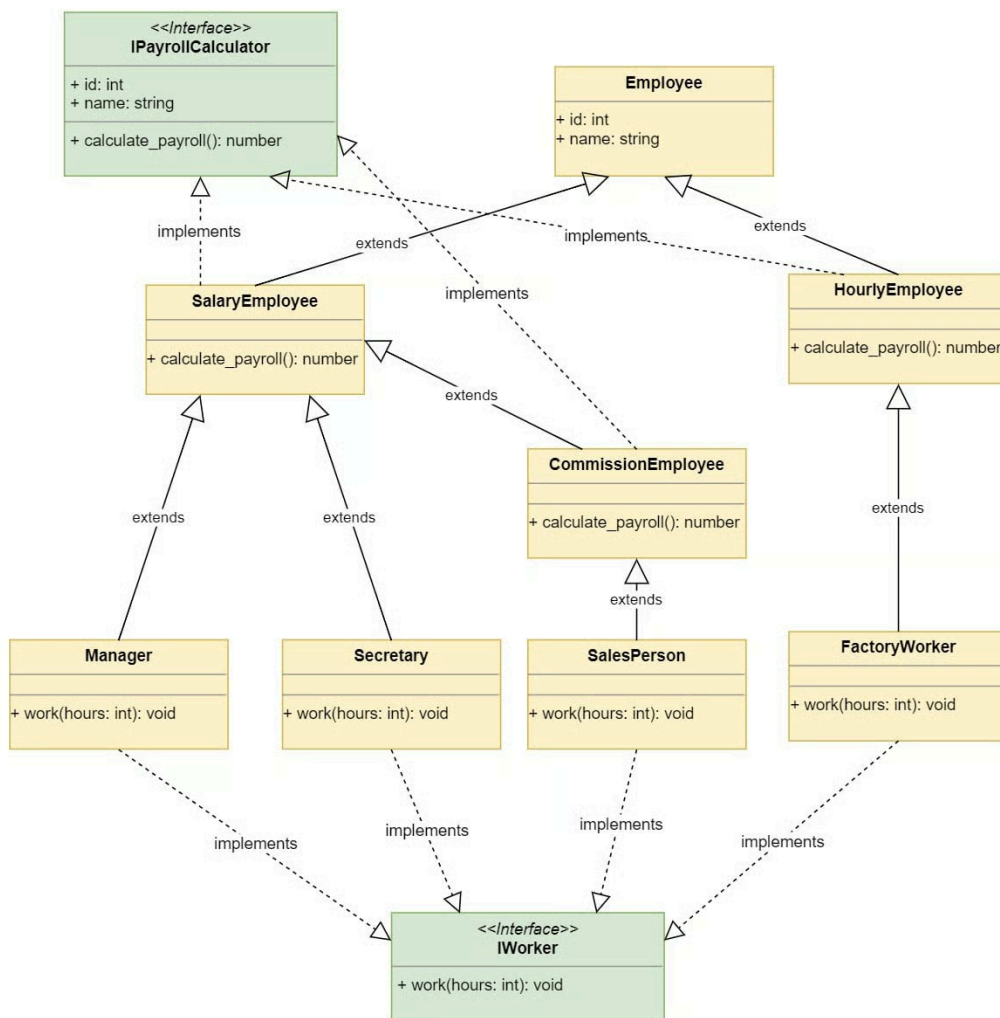
```



for each of the employees.

The program works as expected, but you had to add four new classes to support the changes. As new requirements come, your class hierarchy will inevitably grow, leading to the class explosion problem where your hierarchies will become so big that they'll be hard to understand and maintain.

The following diagram shows the new class hierarchy:



The diagram shows how the class hierarchy is growing. Additional requirements might have an exponential effect on the number of classes with this design.

Remove ads

Inheriting Multiple Classes



derive a class from multiple base classes at the same time.

Multiple inheritance has a bad reputation to the extent that most modern programming languages don't support it. Instead, modern programming languages support the concept of interfaces. In those languages, you inherit from a single base class and then implement multiple interfaces, so you can reuse your classes in different situations.

This approach puts some constraints in your designs. You can only inherit the implementation of one class by directly deriving from it. You can implement multiple interfaces, but you can't inherit the implementation of multiple classes.

This constraint is good for software design because it forces you to design your classes with fewer **dependencies** on each other. You will see later in this tutorial that you can leverage multiple implementations through composition, which makes software more flexible. This section, however, is about multiple inheritance, so take a look at how it works.

It turns out that sometimes temporary secretaries are hired when there's too much paperwork to do. The `TemporarySecretary` class performs the role of a `Secretary` in the context of the `ProductivitySystem`, but for payroll purposes, it's an `HourlyEmployee`.

You look at your class design. It's grown a little bit, but you can still understand how it works. It seems you have two options:

1. **Derive from Secretary:** You can derive from `Secretary` to inherit the `.work()` method for the role, and then override the `.calculate_payroll()` method to implement it as an `HourlyEmployee`.
2. **Derive from HourlyEmployee:** You can derive from `HourlyEmployee` to inherit the `.calculate_payroll()` method,



Then, you remember that Python supports multiple inheritance, so you decide to derive from both `Secretary` and `HourlyEmployee`:

```
Python                                employees.py

# ...

class TemporarySecretary(Secretary, HourlyEmployee):
    pass
```

Python allows you to inherit from two different classes by specifying them between parentheses in the class declaration, and separating them with commas.

Now, you modify your program to add the new temporary secretary employee:

```
Python                                program.py

import hr
import employees
import productivity

manager = employees.Manager(1, "Mary Poppins", 3000)
secretary = employees.Secretary(2, "John Smith", 1500)
sales_guy = employees.SalesPerson(3, "Kevin Bacon", 1000, 1)
factory_worker = employees.FactoryWorker(4, "Jane Doe", 40)
temporary_secretary = employees.TemporarySecretary(5, "Rob", 1000, 1)
company_employees = [
    manager,
    secretary,
    sales_guy,
    factory_worker,
    temporary_secretary,
]

productivity_system = productivity.ProductivitySystem()
productivity_system.track(company_employees, 40)

payroll_system = hr.PayrollSystem()
payroll_system.calculate_payroll(company_employees)
```




Maybe implementing `TemporarySecretary.__init__()` will help:

Python

employees.py

. . .

```
class TemporarySecretary(HourlyEmployee, Secretary):
    def __init__(self, id, name, hours_worked, hourly_rate):
        super().__init__(id, name, hours_worked, hourly_rate)
```

Try it:

Shell



```
$ python program.py
```

Traceback (most recent call last):

```
File "/Users/martin/program.py", line 9, in <module>
    temporary_secretary = employees.TemporarySecretary(5,
                        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/Users/martin/employees.py", line 58, in __init__
    super().__init__(id, name, hours_worked, hourly_rate)
```

```
File "/Users/martin/employees.py", line 18, in __init__
    super().__init__(id, name)
```

```
TypeError: SalaryEmployee.__init__() missing 1 required po
```

That didn't work either. Okay, it's time for you to dive into Python's **method resolution order** (MRO) to see what's going on.

When a method or attribute of a class is accessed, Python uses the class `MRO` to find it. The MRO is also used by `super()` to determine which method or attribute to invoke. You can learn more about `super()` in [Supercharge Your Classes With Python `super\(\)`](#).

You can evaluate the `TemporarySecretary` class MRO using the interactive interpreter:

Python



```
>>> from employees import TemporarySecretary
>>> TemporarySecretary.__mro__
(<class 'employees.TemporarySecretary'>,
```



```
<class 'employees.Employee'>,
<class 'object'>)
```

The MRO shows the order in which Python is going to look for a matching attribute or method. In the example, this is what happens when you create the `TemporarySecretary` object:

1. The `TemporarySecretary.__init__(self, id, name, hours_worked, hourly_rate)` method is called.
2. The `super().__init__(id, name, hours_worked, hourly_rate)` call matches `HourlyEmployee.__init__(self, id, name, hours_worked, hourly_rate)`.
3. `HourlyEmployee` calls `super().__init__(id, name)`, which the MRO is going to match to `Secretary.__init__()`, which is inherited from `SalaryEmployee.__init__(self, id, name, weekly_salary)`.

Because the parameters don't match, Python raises a `TypeError` exception.

You can bypass parts of the MRO. In this case, you want to skip the initialization of `Secretary` and `SalaryEmployee`. You can do this by reversing the inheritance order again back to how you had it initially. Then, you'll directly call `HourlyEmployee.__init__()`:

```
Python                                     employees.py

# ...

class TemporarySecretary(Secretary, HourlyEmployee):
    def __init__(self, id, name, hours_worked, hourly_rate):
        HourlyEmployee.__init__(self, id, name, hours_worked)
```

When you put `Secretary` before `HourlyEmployee`, then the MRO of `TemporarySecretary` looks like the following:

```
Python
```





```
(<class 'employees.TemporarySecretary'>,
 <class 'employees.Secretary'>,
 <class 'employees.SalaryEmployee'>,
 <class 'employees.HourlyEmployee'>,
 <class 'employees.Employee'>,
 <class 'object'>)
```

Because you explicitly specified that `.__init__()` should use `HourlyEmployee.__init__()`, you're effectively skipping `Secretary` and `SalaryEmployee` in the MRO when initializing an object.

That solves the problem of *creating* the object, but you'll run into a similar problem when trying to calculate payroll. You can run the program to see the problem:

Shell



```
$ python program.py
```

```
Tracking Employee Productivity
```

```
=====
```

```
Mary Poppins screams and yells for 40 hours.
```

```
John Smith expends 40 hours doing office paperwork.
```

```
Kevin Bacon expends 40 hours on the phone.
```

```
Jane Doe manufactures gadgets for 40 hours.
```

```
Robin Williams expends 40 hours doing office paperwork.
```

```
Calculating Payroll
```

```
=====
```

```
Payroll for: 1 - Mary Poppins
```

```
- Check amount: 3000
```

```
Payroll for: 2 - John Smith
```

```
- Check amount: 1500
```

```
Payroll for: 3 - Kevin Bacon
```

```
- Check amount: 1250
```

```
Payroll for: 4 - Jane Doe
```

```
- Check amount: 600
```

```
Payroll for: 5 - Robin Williams
```

```
Traceback (most recent call last):
```

```
File "/Users/martin/program.py", line 22, in <module>
```



```

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
File "/Users/martin/employees.py", line 13, in calculate
    return self.weekly_salary
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'TemporarySecretary' object has no attribu

```

The problem now is that because you reversed the inheritance order, the MRO is finding the `.calculate_payroll()` method of `SalariedEmployee` before the one in `HourlyEmployee`. You need to override `.calculate_payroll()` in `TemporarySecretary` and invoke the right implementation from it:

Python

#

```
class TemporarySecretary(Secretary, HourlyEmployee):
    def __init__(self, id, name, hours_worked, hourly_rate):
        HourlyEmployee.__init__(self, id, name, hours_worked, hourly_rate)

    def calculate_payroll(self):
        return HourlyEmployee.calculate_payroll(self)
```

The new `.calculate_payroll()` method now directly invokes `HourlyEmployee.calculate_payroll()` to ensure that you get the correct result. You can run the program again to see it working:

Shell



```
$ python program.py
```

Tracking Employee Productivity

=====

Mary Poppins screams and yells for 40 hours.

John Smith expends 40 hours doing office paperwork.

Kevin Bacon expends 40 hours on the phone.

Jane Doe manufactures gadgets for 40 hours.

Robin Williams expends 40 hours doing office paperwork.

Calculating Payroll

Payroll for: 1 - Mary Poppins



- Check amount: 1500

Payroll for: 3 - Kevin Bacon

- Check amount: 1250

Payroll for: 4 - Jane Doe

- Check amount: 600

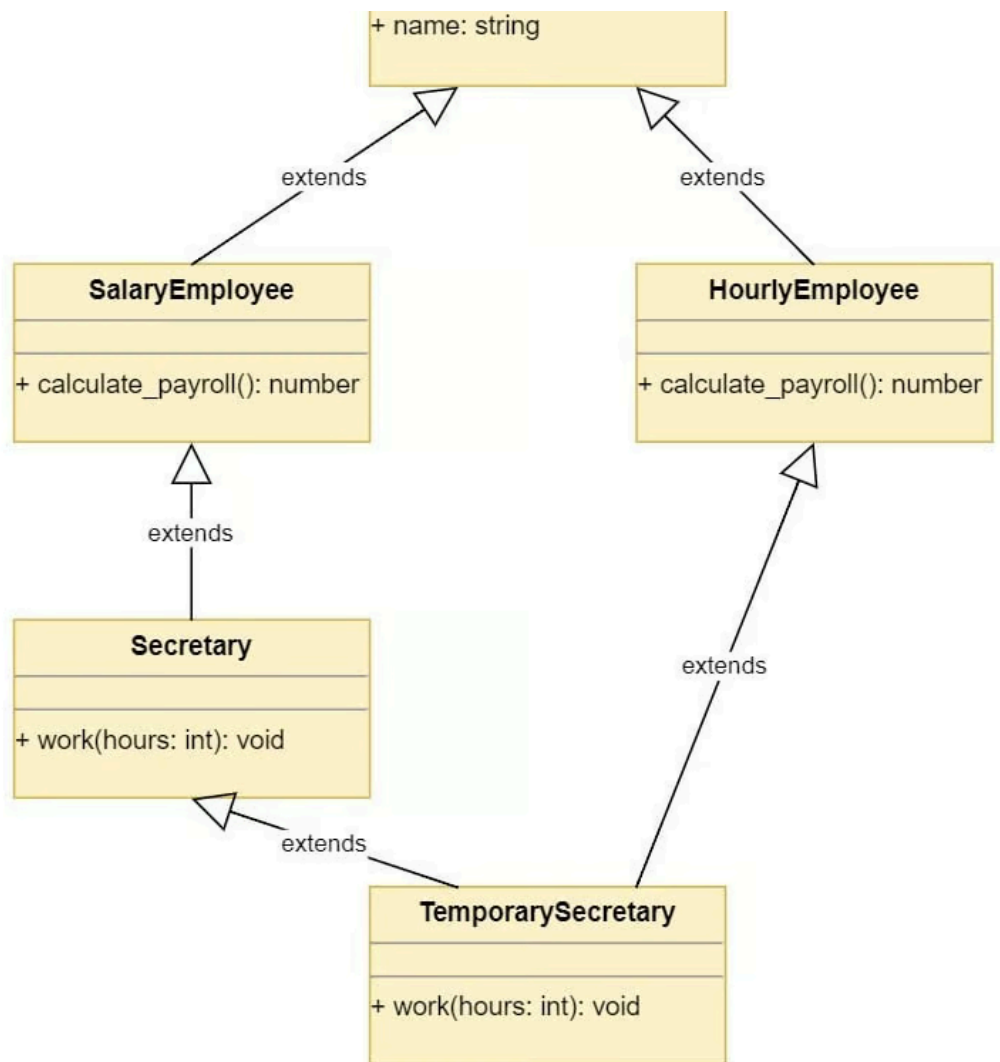
Payroll for: 5 - Robin Williams

- Check amount: 360

The program now works as expected because you're forcing the method resolution order by explicitly telling the interpreter which method you want to use.

As you can see, multiple inheritance can be confusing, especially when you run into the [diamond problem](#).

The following diagram shows the diamond problem in your class hierarchy:



The diagram shows the diamond problem with the current class design. `TemporarySecretary` uses multiple inheritance to derive from two classes that ultimately also derive from `Employee`. This causes two paths to reach the `Employee` base class, which is something you want to avoid in your designs.

The diamond problem appears when you're using multiple inheritance and deriving from two classes that have a common base class. This can cause the wrong version of a method to be called.

As you've seen, Python provides a way to force the right method to be invoked, and analyzing the MRO can help you understand the problem.

Still, when you run into the diamond problem, it's better to rethink the design. You'll now make some changes to leverage multiple



1. **The productivity system** that tracks employee productivity
2. **The payroll system** that calculates the employee payroll

This means that everything related to productivity should be together in one module, and everything related to payroll should be together in another. You can start making changes to the productivity module:

```
Python                                productivity.py

class ProductivitySystem:
    def track(self, employees, hours):
        print("Tracking Employee Productivity")
        print("=====")
        for employee in employees:
            result = employee.work(hours)
            print(f"{employee.name}: {result}")
        print("")

class ManagerRole:
    def work(self, hours):
        return f"screams and yells for {hours} hours."

class SecretaryRole:
    def work(self, hours):
        return f"expends {hours} hours doing office paperw

class SalesRole:
    def work(self, hours):
        return f"expends {hours} hours on the phone."

class FactoryRole:
    def work(self, hours):
        return f"manufactures gadgets for {hours} hours."
```

The productivity module implements the ProductivitySystem class, as well as the related roles that it supports. The classes implement the .work() interface required by the system, but they don't derive from Employee.



```

class PayrollSystem:
    def calculate_payroll(self, employees):
        print("Calculating Payroll")
        print("=====")
        for employee in employees:
            print(f"Payroll for: {employee.id} - {employee.name}")
            print(f"- Check amount: {employee.calculate_payroll()}")
            print("")

class SalaryPolicy:
    def __init__(self, weekly_salary):
        self.weekly_salary = weekly_salary

    def calculate_payroll(self):
        return self.weekly_salary

class HourlyPolicy:
    def __init__(self, hours_worked, hourly_rate):
        self.hours_worked = hours_worked
        self.hourly_rate = hourly_rate

    def calculate_payroll(self):
        return self.hours_worked * self.hourly_rate

class CommissionPolicy(SalaryPolicy):
    def __init__(self, weekly_salary, commission):
        super().__init__(weekly_salary)
        self.commission = commission

    def calculate_payroll(self):
        fixed = super().calculate_payroll()
        return fixed + self.commission

```

The `hr` module implements the `PayrollSystem`, which calculates payroll for the employees. It also implements the policy classes for payroll. As you can see, the policy classes don't derive from `Employee` anymore.

You can now add the necessary classes to the `employee` module:

Python

`employees.py`



```

class Employee:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class Manager(Employee, ManagerRole, SalaryPolicy):
    def __init__(self, id, name, weekly_salary):
        SalaryPolicy.__init__(self, weekly_salary)
        super().__init__(id, name)

class Secretary(Employee, SecretaryRole, SalaryPolicy):
    def __init__(self, id, name, weekly_salary):
        SalaryPolicy.__init__(self, weekly_salary)
        super().__init__(id, name)

class SalesPerson(Employee, SalesRole, CommissionPolicy):
    def __init__(self, id, name, weekly_salary, commission):
        CommissionPolicy.__init__(self, weekly_salary, commission)
        super().__init__(id, name)

class FactoryWorker(Employee, FactoryRole, HourlyPolicy):
    def __init__(self, id, name, hours_worked, hourly_rate):
        HourlyPolicy.__init__(self, hours_worked, hourly_rate)
        super().__init__(id, name)

class TemporarySecretary(Employee, SecretaryRole, HourlyPolicy):
    def __init__(self, id, name, hours_worked, hourly_rate):
        HourlyPolicy.__init__(self, hours_worked, hourly_rate)
        super().__init__(id, name)

```

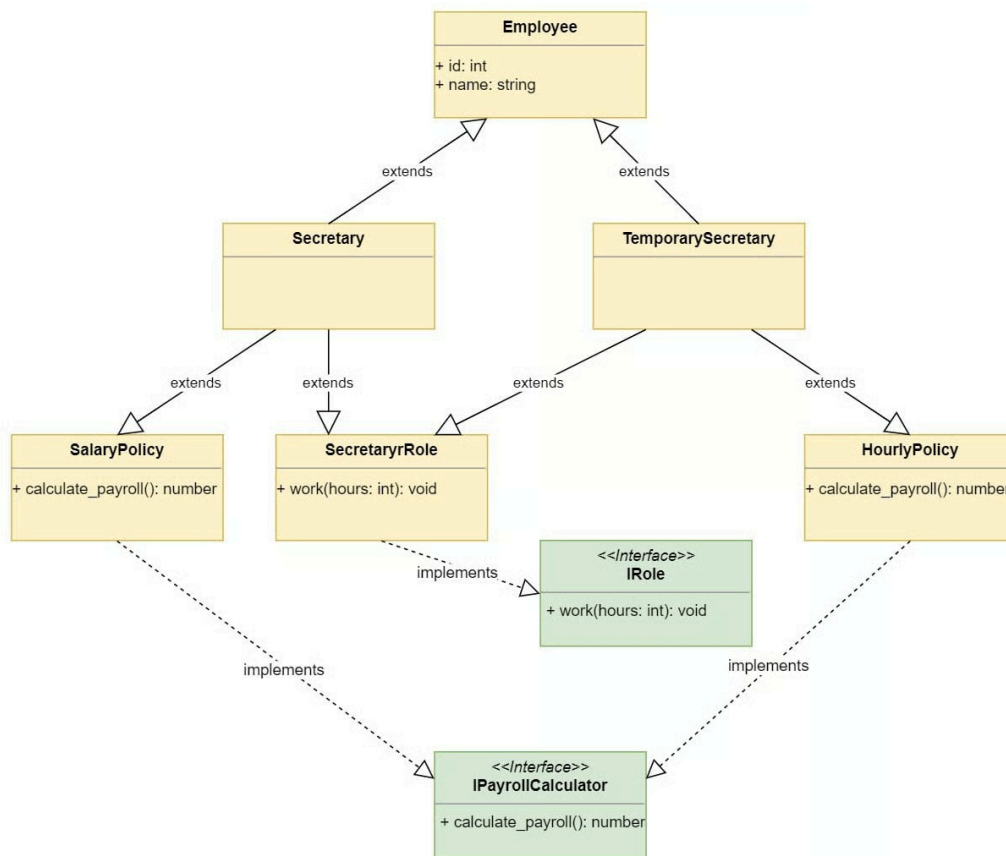
The employees module imports policies and roles from the other modules and implements the different Employee types. You're still using multiple inheritance to inherit the implementation of the salary policy classes and the productivity roles, but the implementation of each class only needs to deal with initialization.

Notice that you still need to explicitly initialize the salary policies in the constructors. You probably saw that the initializations of Manager and Secretary are identical. Also, the initializations of FactoryWorker and TemporarySecretary are the same.



hierarchies.

Here's the UML diagram for the new design:



The diagram shows the relationships to define the **Secretary** and **TemporarySecretary** using multiple inheritance, but avoiding the diamond problem.

You can run the program and see how it works:

Shell



```
$ python program.py
```

```
Tracking Employee Productivity
```

```
=====
```

```
Mary Poppins: screams and yells for 40 hours.
```

```
John Smith: expends 40 hours doing office paperwork.
```

```
Kevin Bacon: expends 40 hours on the phone.
```

```
Jane Doe: manufactures gadgets for 40 hours.
```

```
Robin Williams: expends 40 hours doing office paperwork.
```

```
Calculating Payroll
```

```
=====
```



```
Payroll for: 2 - John Smith  
- Check amount: 1500
```

```
Payroll for: 3 - Kevin Bacon  
- Check amount: 1250
```

```
Payroll for: 4 - Jane Doe  
- Check amount: 600
```

```
Payroll for: 5 - Robin Williams  
- Check amount: 360
```

You've seen how inheritance and multiple inheritance work in Python. You can now explore the topic of composition.

 Remove ads

Composition in Python

Composition is an object-oriented design concept that models a **has a** relationship. In composition, a class known as **composite** contains an object, or **component**, of another class. In other words, a composite class **has a** component of another class.

Composition allows composite classes to reuse the implementation of the components it contains. The composite class doesn't inherit the component class interface, but it can leverage its implementation.

The composition relation between two classes is considered loosely coupled. That means that changes to the component class rarely affect the composite class, and changes to the composite class never affect the component class.

This provides better adaptability to change and allows applications to introduce new requirements without affecting existing code.

When looking at two competing software designs, one based on inheritance and another based on composition, the composition



You've already used composition in your examples. If you look at the `Employee` class, then you'll see that it contains two attributes:

1. `.id` to identify an employee
2. `.name` to contain the name of the employee

These two attributes are objects that the `Employee` class has.

Therefore, you can say that an `Employee` **has an** `.id` and **has a** `.name`.

Another attribute for an `Employee` might be an `Address`. Create a new Python file called `contacts.py` and add code for an `Address` class:

```
Python                                contacts.py

class Address:
    def __init__(self, street, city, state, zipcode, street2):
        self.street = street
        self.street2 = street2
        self.city = city
        self.state = state
        self.zipcode = zipcode

    def __str__(self):
        lines = [self.street]
        if self.street2:
            lines.append(self.street2)
        lines.append(f"{self.city}, {self.state} {self.zipcode}")
        return "\n".join(lines)
```

You implemented a basic address class that contains the usual components for an address. You made the `street2` attribute optional because not all addresses will have that component.

You implemented `__str__()` to provide a pretty representation of an `Address`. You can see this implementation in the interactive interpreter:

```
Python                                

>>> from contacts import Address
>>> address = Address("55 Main St.", "Concord", "NH", "03301", "Apt 101")
```



When you `print()` the address variable, you're invoking the **special method** `__str__()`. Since you overloaded the method to return a string formatted as an address, you get a nice, readable representation. [Operator and Function Overloading in Custom Python Classes](#) gives a good overview of the special methods available in classes that you can implement to customize the behavior of your objects.

You can now add Address to the Employee class through composition:

```
Python                                     employees.py

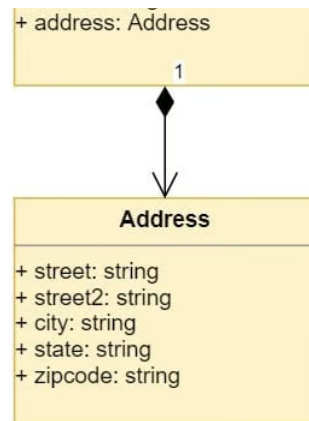
# ...

class Employee:
    def __init__(self, id, name):
        self.id = id
        self.name = name
        self.address = None
```

You initialize the `.address` attribute to `None` for now to make it optional, but by doing that, you can now assign an Address to an Employee. Also notice that there's no reference in the `employee` module to the `contacts` module.

Composition is a loosely coupled relationship that often doesn't require the composite class to have knowledge of the component.

The UML diagram representing the relationship between Employee and Address looks like this:



The diagram shows the basic composition relationship between Employee and Address.

You can now modify the PayrollSystem class to leverage the .address attribute in Employee:

Python

hr.py

```

class PayrollSystem:
    def calculate_payroll(self, employees):
        print("Calculating Payroll")
        print("=====")
        for employee in employees:
            print(f"Payroll for: {employee.id} - {employee.name}")
            print(f"- Check amount: {employee.calculate_payroll()}")
            if employee.address:
                print("- Sent to:")
                print(employee.address)
            print("")
  
```

You check to see if the employee object has an address, and if it does, you print it. You can now modify the program to assign some addresses to the employees:

Python

program.py

```

import hr
import employees
import productivity
import contacts

manager = employees.Manager(1, "Mary Poppins", 3000)
  
```



```

secretary = employees.Secretary(2, "John Smith", 1500)
secretary.address = contacts.Address(
    "67 Paperwork Ave.", "Manchester", "NH", "03101"
)
sales_guy = employees.SalesPerson(3, "Kevin Bacon", 1000, 40)
factory_worker = employees.FactoryWorker(4, "Jane Doe", 40)
temporary_secretary = employees.TemporarySecretary(5, "Rob
employees = [
    manager,
    secretary,
    sales_guy,
    factory_worker,
    temporary_secretary,
]

productivity_system = productivity.ProductivitySystem()
productivity_system.track(employees, 40)

payroll_system = hr.PayrollSystem()
payroll_system.calculate_payroll(employees)

```

You added a couple of addresses to the manager and secretary objects. When you run the program, you'll see the addresses printed:

Shell



```
$ python program.py
```

```
Tracking Employee Productivity
```

```
=====
```

```
Mary Poppins: screams and yells for 40 hours.
```

```
John Smith: expends 40 hours doing office paperwork.
```

```
Kevin Bacon: expends 40 hours on the phone.
```

```
Jane Doe: manufactures gadgets for 40 hours.
```

```
Robin Williams: expends 40 hours doing office paperwork.
```

```
Calculating Payroll
```

```
=====
```

```
Payroll for: 1 - Mary Poppins
```

```
- Check amount: 3000
```

```
- Sent to:
```

```
121 Admin Rd
```

```
Concord, NH 03301
```



67 Paperwork Ave.
Manchester, NH 03101

Payroll for: 3 - Kevin Bacon
- Check amount: 1250

Payroll for: 4 - Jane Doe
- Check amount: 600

Payroll for: 5 - Robin Williams
- Check amount: 360

Notice how the payroll output for the manager and secretary objects shows the addresses where the checks were sent.

The `Employee` class leverages the implementation of the `Address` class without any knowledge of what an `Address` object is or how it's represented. This type of design is so flexible that you can change the `Address` class without any impact to the `Employee` class.

 Remove ads

Flexible Designs With Composition

Composition is more flexible than inheritance because it models a loosely coupled relationship. Changes to a component class have minimal or no effects on the composite class. Designs based on composition are more suitable to change.

You change behavior by providing new components that implement those behaviors instead of adding new classes to your hierarchy.

Take a look at the multiple inheritance example above. Imagine how new payroll policies will affect the design. Try to picture what the class hierarchy will look like if new roles are needed. As you saw before, relying too heavily on inheritance can lead to class explosion.

The biggest problem isn't so much the number of classes in your design, but how tightly coupled the relationships between those



In this section, you're going to use composition to implement a better design that still fits the requirements of the `PayrollSystem` and the `ProductivitySystem`.

You can start by implementing the functionality of the `ProductivitySystem`:

```
Python                                productivity.py

class ProductivitySystem:
    def __init__(self):
        self._roles = {
            "manager": ManagerRole,
            "secretary": SecretaryRole,
            "sales": SalesRole,
            "factory": FactoryRole,
        }

    def get_role(self, role_id):
        role_type = self._roles.get(role_id)
        if not role_type:
            raise ValueError(role_id)
        return role_type()

    def track(self, employees, hours):
        print("Tracking Employee Productivity")
        print("=====")
        for employee in employees:
            employee.work(hours)
        print("")
```

The updated `ProductivitySystem` class defines some roles using a string identifier mapped to a role class that implements the role. It exposes a `.get_role()` method that, given a role identifier, returns the role type object. If the role isn't found, then Python raises a `ValueError` exception.

It also exposes the previous functionality in the `.track()` method, where given a list of employees, it tracks the productivity of those employees.



```
# ...

class ManagerRole:
    def perform_duties(self, hours):
        return f"screams and yells for {hours} hours."

class SecretaryRole:
    def perform_duties(self, hours):
        return f"does paperwork for {hours} hours."

class SalesRole:
    def perform_duties(self, hours):
        return f"expends {hours} hours on the phone."

class FactoryRole:
    def perform_duties(self, hours):
        return f"manufactures gadgets for {hours} hours."
```

Each of the roles that you implemented exposes its own `.perform_duties()` method that takes the number of hours worked. These methods return a string representing the duties.

Note: If you’ve followed along throughout the section on inheritance, then you’ll notice that these roles are similar, but slightly different from that example.

Feel free to continue working with the roles that you previously defined and their `.work()` methods if you prefer. You’ll just need to adapt the relevant names to account for the change.

The role classes are independent of each other, but they expose the same interface, so they’re interchangeable. You’ll see later how they’re used in the application.

Now, you can implement the `PayrollSystem` for the application:

Python

hr.py



```

self._employee_policies = {
    1: SalaryPolicy(3000),
    2: SalaryPolicy(1500),
    3: CommissionPolicy(1000, 100),
    4: HourlyPolicy(15),
    5: HourlyPolicy(9),
}

def get_policy(self, employee_id):
    policy = self._employee_policies.get(employee_id)
    if not policy:
        return ValueError(employee_id)
    return policy

def calculate_payroll(self, employees):
    print("Calculating Payroll")
    print("=====")
    for employee in employees:
        print(f"Payroll for: {employee.id} - {employee}")
        print(f"- Check amount: {employee.calculate_payroll}")
        if employee.address:
            print("- Sent to:")
            print(employee.address)
        print("")

```

PayrollSystem keeps an internal database of payroll policies for each employee. It exposes a `.get_policy()` method that, given an employee `.id`, returns its payroll policy. If a specified `.id` doesn't exist in the system, then the method raises a `ValueError` exception.

The implementation of `.calculate_payroll()` works the same as before. It takes a list of employees, calculates the payroll, and prints the results.

You can now implement the payroll policy classes:

```

Python                                     hr.py

# ...

class PayrollPolicy:
    def __init__(self):
        self.hours_worked = 0

```



```
class SalaryPolicy(PayrollPolicy):
    def __init__(self, weekly_salary):
        super().__init__()
        self.weekly_salary = weekly_salary

    def calculate_payroll(self):
        return self.weekly_salary

class HourlyPolicy(PayrollPolicy):
    def __init__(self, hourly_rate):
        super().__init__()
        self.hourly_rate = hourly_rate

    def calculate_payroll(self):
        return self.hours_worked * self.hourly_rate

class CommissionPolicy(SalaryPolicy):
    def __init__(self, weekly_salary, commission_per_sale):
        super().__init__(weekly_salary)
        self.commission_per_sale = commission_per_sale

    @property
    def commission(self):
        sales = self.hours_worked / 5
        return sales * self.commission_per_sale

    def calculate_payroll(self):
        fixed = super().calculate_payroll()
        return fixed + self.commission
```

You first implement a `PayrollPolicy` class that serves as a base class for all the payroll policies. This class tracks the `hours_worked`, which is common to all payroll policies.

The other policy classes derive from `PayrollPolicy`. You use inheritance here because you want to leverage the implementation of `PayrollPolicy`. Also, `SalaryPolicy`, `HourlyPolicy`, and `CommissionPolicy` **are a** `PayrollPolicy`.

`SalaryPolicy` is initialized with a `weekly_salary` value that `.calculate_payroll()` then uses. `HourlyPolicy` is initialized with



The `CommissionPolicy` class derives from `SalaryPolicy` because it wants to inherit its implementation. It's initialized with the `weekly_salary` parameters, but it also requires a `commission_per_sale` parameter.

The `.commission_per_sale` is used to calculate the `.commission`, which is implemented as a property so it gets calculated when requested. In the example, you're assuming that a sale happens every five hours worked, and the `.commission` is the number of sales times the `.commission_per_sale` value.

`CommissionPolicy` implements the `.calculate_payroll()` method by first leveraging the implementation in `SalaryPolicy` and then adding the calculated commission.

You can now add an `AddressBook` class to manage employee addresses:

```
Python                                contacts.py

# ...

class AddressBook:
    def __init__(self):
        self._employee_addresses = {
            1: Address("121 Admin Rd.", "Concord", "NH", "03301"),
            2: Address("67 Paperwork Ave", "Manchester", "NH", "03101"),
            3: Address("15 Rose St", "Concord", "NH", "03301"),
            4: Address("39 Sole St.", "Concord", "NH", "03301"),
            5: Address("99 Mountain Rd.", "Concord", "NH", "03301"),
        }

    def get_employee_address(self, employee_id):
        address = self._employee_addresses.get(employee_id)
        if not address:
            raise ValueError(employee_id)
        return address
```

The `AddressBook` class keeps an internal database of `Address` objects for each employee. It exposes a `.get_employee_address()`



The Address class implementation remains the same as before:

Python

contacts.py

```
class Address:
    def __init__(self, street, city, state, zipcode, street2):
        self.street = street
        self.street2 = street2
        self.city = city
        self.state = state
        self.zipcode = zipcode

    def __str__(self):
        lines = [self.street]
        if self.street2:
            lines.append(self.street2)
        lines.append(f"{self.city}, {self.state} {self.zipcode}")
        return "\n".join(lines)
```

The class manages the address components and provides a pretty representation of an address.

So far, the new classes have been extended to support more functionality, but there are no significant changes to the previous design. This is going to change with the design of the employees module and its classes.

You can start by implementing an EmployeeDatabase class:

Python

employees.py

```
from productivity import ProductivitySystem
from hr import PayrollSystem
from contacts import AddressBook

class EmployeeDatabase:
    def __init__(self):
        self._employees = [
            {"id": 1, "name": "Mary Poppins", "role": "manager"},
            {"id": 2, "name": "John Smith", "role": "secretary"},
            {"id": 3, "name": "Kevin Bacon", "role": "sales"},
            {"id": 4, "name": "Jane Doe", "role": "factory"},
            {"id": 5, "name": "Robin Williams", "role": "sales"}]
```



```

self.employee_addresses = AddressBook()

@property
def employees(self):
    return [self._create_employee(**data) for data in

def _create_employee(self, id, name, role):
    address = self.employee_addresses.get_employee_add
    employee_role = self.productivity.get_role(role)
    payroll_policy = self.payroll.get_policy(id)
    return Employee(id, name, address, employee_role,

```

EmployeeDatabase keeps track of all the employees in the company. For each employee, it tracks the `.id`, `.name`, and `.role`. It **has an** instance of the `ProductivitySystem`, the `PayrollSystem`, and the `AddressBook`. These instances are used to create employees.

It exposes an `.employees` property that returns the list of employees. The `Employee` objects are created in an internal `._create_employee()` method. Notice that you don't have different types of `Employee` classes. You just need to implement a single `Employee` class:

```

Python                                     employees.py

# ...

class Employee:
    def __init__(self, id, name, address, role, payroll):
        self.id = id
        self.name = name
        self.address = address
        self.role = role
        self.payroll = payroll

    def work(self, hours):
        duties = self.role.perform_duties(hours)
        print(f"Employee {self.id} - {self.name}:")
        print(f"- {duties}")
        print("")
        self.payroll.track_work(hours)

```

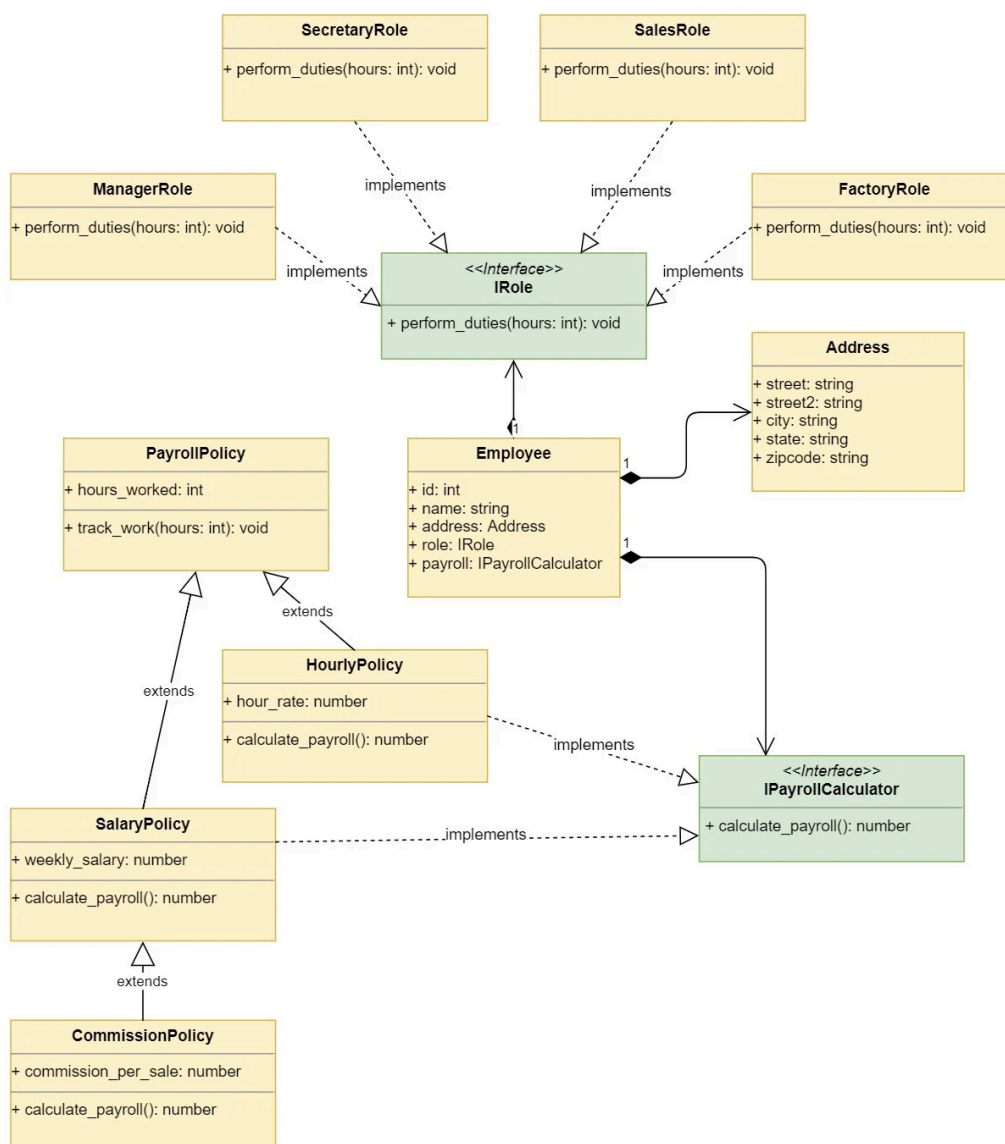


You initialize the `Employee` class with the `.id`, `.name`, and `.address` attributes. This class also requires the productivity `.role` for the employee and the `.payroll` policy.

The class exposes a `.work()` method that takes the hours worked. This method first retrieves the duties from the `.role`. In other words, it delegates to the `.role` object to perform its duties.

In the same way, it delegates to the `.payroll` object to track the work hours. The `.payroll`, as you saw, uses those hours to calculate the payroll if needed.

The following diagram shows the composition design used:





Address and depends on the `IRole` and `IPayrollCalculator` interfaces to delegate the work. There are multiple implementations of these interfaces.

You can now use this design in your program:

```
Python                                program.py

from hr import PayrollSystem
from productivity import ProductivitySystem
from employees import EmployeeDatabase

productivity_system = ProductivitySystem()
payroll_system = PayrollSystem()
employee_database = EmployeeDatabase()

employees = employee_database.employees
productivity_system.track(employees, 40)
payroll_system.calculate_payroll(employees)
```

You can run the program to see its output:

Shell



```
$ python program.py
```

```
Tracking Employee Productivity
```

```
=====
```

```
Employee 1 - Mary Poppins:
```

```
- screams and yells for 40 hours.
```

```
Employee 2 - John Smith:
```

```
- does paperwork for 40 hours.
```

```
Employee 3 - Kevin Bacon:
```

```
- expends 40 hours on the phone.
```

```
Employee 4 - Jane Doe:
```

```
- manufactures gadgets for 40 hours.
```

```
Employee 5 - Robin Williams:
```

```
- does paperwork for 40 hours.
```

```
Calculating Payroll
```



- Sent to:
121 Admin Rd.
Concord, NH 03301

Payroll for: 2 - John Smith
- Check amount: 1500
- Sent to:
67 Paperwork Ave
Manchester, NH 03101

Payroll for: 3 - Kevin Bacon
- Check amount: 1800.0
- Sent to:
15 Rose St
Apt. B-1
Concord, NH 03301

Payroll for: 4 - Jane Doe
- Check amount: 600
- Sent to:
39 Sole St.
Concord, NH 03301

Payroll for: 5 - Robin Williams
- Check amount: 360
- Sent to:
99 Mountain Rd.
Concord, NH 03301

This design is what's called [policy-based design](#), where classes are composed of policies, and they delegate to those policies to do the work.

Policy-based design was introduced in the book [Modern C++ Design](#), and it uses template metaprogramming in C++ to achieve the results.

Python doesn't support templates, but you can achieve similar results using composition, as you saw in the example above.

This type of design gives you all the flexibility you'll need as requirements change. Imagine that you need to change the way payroll is calculated for an object at runtime.



Customizing Behavior With Composition

If your design relies on inheritance, then you need to find a way to change the type of an object to change its behavior. With composition, you just need to change the policy that the object uses.

Imagine that your manager all of a sudden becomes a temporary employee who gets paid by the hour. You can modify the object during the execution of the program in the following way:

```
Python                                program.py

from hr import PayrollSystem, HourlyPolicy
from productivity import ProductivitySystem
from employees import EmployeeDatabase

productivity_system = ProductivitySystem()
payroll_system = PayrollSystem()
employee_database = EmployeeDatabase()

employees = employee_database.employees
manager = employees[0]
manager.payroll = HourlyPolicy(55)

productivity_system.track(employees, 40)
payroll_system.calculate_payroll(employees)
```

The program gets the employee list from the `EmployeeDatabase` and retrieves the first employee, which is the manager you want. Then it creates a new `HourlyPolicy` initialized at 55 dollars per hour and assigns it to the manager object.

The new policy is now used by the `PayrollSystem`, modifying the existing behavior. You can run the program again to see the result:

Shell



```
$ python program.py
```

```
Tracking Employee Productivity
=====
Employee 1 - Mary Poppins:
- screams and yells for 40 hours.
```



Employee 3 - Kevin Bacon:

- expends 40 hours on the phone.

Employee 4 - Jane Doe:

- manufactures gadgets for 40 hours.

Employee 5 - Robin Williams:

- does paperwork for 40 hours.

Calculating Payroll

=====

Payroll for: 1 - Mary Poppins

- Check amount: 2200

- Sent to:

121 Admin Rd.

Concord, NH 03301

Payroll for: 2 - John Smith

- Check amount: 1500

- Sent to:

67 Paperwork Ave

Manchester, NH 03101

Payroll for: 3 - Kevin Bacon

- Check amount: 1800.0

- Sent to:

15 Rose St

Apt. B-1

Concord, NH 03301

Payroll for: 4 - Jane Doe

- Check amount: 600

- Sent to:

39 Sole St.

Concord, NH 03301

Payroll for: 5 - Robin Williams

- Check amount: 360

- Sent to:

99 Mountain Rd.

Concord, NH 03301



nave.

Notice how you added that business rule to the program without changing any of the existing classes. Consider what type of changes would've been required with an inheritance design.

You would've had to create a new class and change the type of the manager employee. There's no chance that you could've changed the policy at runtime.

Choosing Between Inheritance and Composition in Python

So far, you've seen how inheritance and composition work in Python. You've seen that derived classes inherit the interface and implementation of their base classes. You've also seen that composition allows you to reuse the implementation of another class.

You've implemented two solutions to the same problem. The first solution used multiple inheritance, and the second one used composition.

You've also seen that Python's duck typing allows you to reuse objects with existing parts of a program by implementing the desired interface. In Python, it isn't necessary to derive from a base class to reuse your classes.

At this point, you might be asking when to use inheritance vs composition in Python. They both enable code reuse. Inheritance and composition can tackle similar problems in your Python programs.

The general advice is to use the relationship that creates fewer dependencies between two classes. This relation is composition. Still, there'll be times where inheritance will make more sense.

The following sections provide some guidelines to help you make the right choice between inheritance and composition in Python.



Liskov's substitution principle says that an object of type `Derived`, which inherits from `Base`, can replace an object of type `Base` without altering the desirable properties of a program.

Liskov's substitution principle is the most important guideline to determine if inheritance is the appropriate design solution. Still, the answer might not be straightforward in all situations. Fortunately, there's a simple test that you can use to determine if your design follows Liskov's substitution principle.

Let's say you have a class, `A`, that provides an implementation and interface you want to reuse in another class, `B`. Your initial thought is that you can derive `B` from `A` and inherit both the interface and the implementation. To be sure this is the right design, you follow these steps:

1. **Evaluate `B` is an `A`:** Think about this relationship and justify it. Does it make sense?
2. **Evaluate `A` is a `B`:** Reverse the relationship and justify it. Does it also make sense?

If you can justify both relationships, then you should never inherit those classes from one another. Look at a more concrete example.

You have a `Rectangle` class that exposes an `.area` property. You need a `Square` class, which also has an `.area`. It seems that a `Square` is a special type of `Rectangle`, so maybe you can derive from it and leverage both the interface and implementation.

Before you jump into the implementation, you use Liskov's substitution principle to evaluate the relationship.

A `Square` **is a** `Rectangle` because its area is calculated from the product of its `.height` times its `.length`. The constraint is that `Square.height` and `Square.length` must be equal.



makes sense.

A `Rectangle` **is a** `Square` because its area is calculated from the product of its `.height` times its `.length`. The difference is that `Rectangle.height` and `Rectangle.width` can change independently.

It also makes sense. You can justify the relationship and describe the special constraints for each class. This is a good sign that these two classes should never derive from each other.

You might have seen other examples that derive `Square` from `Rectangle` to explain inheritance. You might be skeptical with the little test that you just did. Fair enough. Next, you'll write a program that illustrates the problem with deriving `Square` from `Rectangle`.

First, you implement `Rectangle`. You're even going to [encapsulate](#) the attributes to ensure that you're meeting all the constraints:

Python `rectangle_square_demo.py`

```
class Rectangle:
    def __init__(self, length, height):
        self._length = length
        self._height = height

    @property
    def area(self):
        return self._length * self._height
```

You initialize the `Rectangle` class with a `length` and a `height`, and the class provides an `.area` property that returns the area. The `length` and `height` are encapsulated as `._length` and `._height` to avoid changing them directly.

Now, you derive `Square` from `Rectangle` and override the necessary interface to meet the constraints of a `Square`:

Python `rectangle_square_demo.py`



```
class Square(Rectangle):  
    def __init__(self, side_size):  
        super().__init__(side_size, side_size)
```

You initialize the Square class with a `side_size`, which is used to initialize both components of the base class. Now, you write a small program to test the behavior:

```
Python                                rectangle_square_demo.py  
  
# ...  
  
rectangle = Rectangle(2, 4)  
assert rectangle.area == 8  
  
square = Square(2)  
assert square.area == 4  
  
print("OK!")
```

The program creates a `Rectangle` and a `Square` and asserts that their `.area` is calculated correctly. You can run the program and see that everything is OK so far:

Shell



```
$ python rectangle_square_demo.py
```

```
OK!
```

The program executes correctly, so it seems that `Square` is just a special case of a `Rectangle`.

Later on, you need to support resizing `Rectangle` objects, so you make the appropriate changes to the class:

```
Python                                rectangle_square_demo.py  
  
class Rectangle:  
    def __init__(self, length, height):  
        self._length = length  
        self._height = height
```



```
        return self._length * self._height

    def resize(self, new_length, new_height):
        self._length = new_length
        self._height = new_height
```

Your `.resize()` method takes the `new_length` and `new_width` for the object. You can add the following code to the program to verify that it works correctly:

```
Python                                rectangle_square_demo.py

# ...

rectangle.resize(3, 5)
assert rectangle.area == 15

print("OK!")
```

You resize the rectangle object and assert that the new area is correct. You can run the program to verify the behavior:

```
Shell                                

$ python rectangle_square_demo.py

OK!
```

The assertion passes, and you see that the program runs correctly.

So, what happens if you resize a square? Modify the program, and try to modify the square object:

```
Python                                rectangle_square_demo.py

# ...

square.resize(3, 5)
print(f"Square area: {square.area}")

print("OK!")
```



see:

Shell



```
$ python rectangle_square_demo.py
```

```
Square area: 15
```

```
OK!
```

The program shows that the new area is 15 like the `rectangle` object. The problem now is that the square object no longer meets the `Square` class constraint that the length and height must be equal.

How can you fix that problem? You can try several approaches, but all of them will be awkward. You can override `.resize()` in `Square` and ignore the height parameter. However, that will be confusing for people looking at other parts of the program where `Rectangle` objects are being resized and some of them are not getting the expected areas because they're really `Square` objects.

In a small program like this one, it might be easy to spot the causes of the weird behavior, but in a more complex program, the problem will be harder to find.

The reality is that if you're able to justify an inheritance relationship between two classes both ways, then you shouldn't derive one class from another.

In the example, it doesn't make sense that `Square` inherits the interface and implementation of `.resize()` from `Rectangle`. That doesn't mean that `Square` objects can't be resized. It means that the interface is different because it only needs a `side_size` parameter.

This difference in interface justifies not deriving `Square` from `Rectangle`, like the test above advised.

 Remove ads

Mixing Features With Mixin Classes



other classes but isn't considered a base class.

A mixin allows other classes to reuse its interface and implementation without becoming a superclass. It implements a unique behavior that you can aggregate to other unrelated classes. Mixins are similar to composition, but they create a stronger relationship.

Say you want to convert objects of certain types in your application to a **dictionary** representation of the object. You could provide a `.to_dict()` method in every class that you want to support this feature, but the implementation of `.to_dict()` seems to be very similar.

This could be a good candidate for a mixin. You start by slightly modifying the `Employee` class from the composition example:

```
Python                                     employees.py

# ...

class Employee:
    def __init__(self, id, name, address, role, payroll):
        self.id = id
        self.name = name
        self.address = address
        self._role = role
        self._payroll = payroll

    def work(self, hours):
        duties = self._role.perform_duties(hours)
        print(f"Employee {self.id} - {self.name}:")
        print(f"- {duties}")
        print("")
        self._payroll.track_work(hours)

    def calculate_payroll(self):
        return self._payroll.calculate_payroll()
```

The changes are minimal. You just changed the `.role` and `.payroll` attributes to be internal by adding a **leading underscore** to their



representations.py:

```
Python                                representations.py

class AsDictionaryMixin:
    def to_dict(self):
        return {
            prop: self._represent(value)
            for prop, value in self.__dict__.items()
            if not self._is_internal(prop)
        }

    def _represent(self, value):
        if isinstance(value, object):
            if hasattr(value, "to_dict"):
                return value.to_dict()
            else:
                return str(value)
        else:
            return value

    def _is_internal(self, prop):
        return prop.startswith("_")
```

The `AsDictionaryMixin` class exposes a `.to_dict()` method that returns the representation of itself as a dictionary. The method is implemented as a **dict comprehension** that creates a dictionary mapping `prop` to `value` for each item in `self.__dict__.items()` if the `prop` isn't internal.

Note: This is why you made the `role` and `payroll` attributes internal in the `Employee` class—because you don't want to represent them in the dictionary.

As you saw at the beginning, creating a class inherits some members from `object`, and one of those members is `.__dict__`, which is basically a mapping of all the attributes in an object to their values.



```
._is_internal().
```

With `._represent()`, you check the specified value. If the value **is an** object, then the method looks to see if it also has a `._to_dict()` member and uses it to represent the object. Otherwise, it returns a string representation. If the value isn't an object, then it simply returns the value.

You can modify the `Employee` class to support this mixin:

```
Python                                employees.py

# ...

from representations import AsDictionaryMixin

# ...

class Employee(AsDictionaryMixin):
    # ...
```

All you have to do is inherit the `AsDictionaryMixin` to support the functionality. It'll be nice to support the same functionality in the `Address` class, so you represent the `Employee.address` attribute in the same way:

```
Python                                contacts.py

from representations import AsDictionaryMixin

class Address(AsDictionaryMixin):
    # ...
```

You apply the mixin to the `Address` class to support the feature. Now, you can write a small program to test it:

```
Python                                program.py

import json

from employees import EmployeeDatabase
```



```
for employee in EmployeeDatabase().employees:  
    print_dict(employee.to_dict())
```

The program implements `print_dict()`, which converts the dictionary to a **JSON** string using indentation so the output looks better.

Then, it iterates through all the employees, printing the dictionary representation provided by `.to_dict()`. You can run the program to see its output:

Shell



```
$ python program.py  
  
{  
  "id": "1",  
  "name": "Mary Poppins",  
  "address": {  
    "street": "121 Admin Rd.",  
    "street2": "",  
    "city": "Concord",  
    "state": "NH",  
    "zipcode": "03301"  
  }  
}  
{  
  "id": "2",  
  "name": "John Smith",  
  "address": {  
    "street": "67 Paperwork Ave",  
    "street2": "",  
    "city": "Manchester",  
    "state": "NH",  
    "zipcode": "03101"  
  }  
}  
{  
  "id": "3",  
  "name": "Kevin Bacon",  
  "address": {  
    "street": "15 Rose St",  
    "street2": "Apt. B-1",
```



```
}  
}  
{  
    "id": "4",  
    "name": "Jane Doe",  
    "address": {  
        "street": "39 Sole St.",  
        "street2": "",  
        "city": "Concord",  
        "state": "NH",  
        "zipcode": "03301"  
    }  
}  
{  
    "id": "5",  
    "name": "Robin Williams",  
    "address": {  
        "street": "99 Mountain Rd.",  
        "street2": "",  
        "city": "Concord",  
        "state": "NH",  
        "zipcode": "03301"  
    }  
}
```

You leveraged the implementation of `AsDictionaryMixin` in both `Employee` and `Address` classes even when they're not related. Because `AsDictionaryMixin` only provides behavior, you can reuse it with other classes without causing problems.

Composition to Model “Has A” Relationship

Composition models a **has a** relationship. With composition, a class `Composite` **has an** instance of the class `Component` and can leverage its implementation. You can reuse the `Component` class in other classes completely unrelated to the `Composite`.

In the composition example above, the `Employee` class **has an** `Address` object. `Address` implements all the functionality to handle addresses, and other classes can reuse it.



implementation, ensuring that addresses are handled consistently across the application.

A problem that you may run into when using composition is that some of your classes may start growing by using multiple components. Your classes may require multiple parameters in the constructor just to pass in the components that they're made of. This can make your classes hard to use.

A way to avoid the problem is by using the [factory method](#) to construct your objects. You did that with the composition example.

If you look at the implementation of the `EmployeeDatabase` class, then you'll notice that it uses `._create_employee()` to construct an `Employee` object with the right parameters.

This design will work, but ideally, you should be able to construct an `Employee` object just by specifying an ID, for example `employee = Employee(1)`.

The following changes might improve your design. You can start with the `productivity` module:

```
Python                                productivity.py

class _ProductivitySystem:
    # ...

# ...

_productivity_system = _ProductivitySystem()

def get_role(role_id):
    return _productivity_system.get_role(role_id)

def track(employees, hours):
    _productivity_system.track(employees, hours)
```

First, you make the `_ProductivitySystem` class internal by prepending an underscore to the class name. Then you provide a `_productivity_system` internal variable to the module. You're



`get_role()` and `track()`, as the public interface to the module. This is what other modules should use.

What you're saying is that `_ProductivitySystem` is a [singleton](#), and there should only be one object created from it.

Now, you can do the same with the `hr` module:

```
Python                                hr.py

class _PayrollSystem:
    # ...

# ...

_payroll_system = _PayrollSystem()

def get_policy(employee_id):
    return _payroll_system.get_policy(employee_id)

def calculate_payroll(employees):
    _payroll_system.calculate_payroll(employees)
```

Again, you make `_PayrollSystem` internal and provide a public interface to it. The application will use the public interface to get policies and calculate payroll.

You'll now do the same with the `contacts` module:

```
Python                                contacts.py

# ...

class _AddressBook:
    # ...

_address_book = _AddressBook()

def get_employee_address(employee_id):
    return _address_book.get_employee_address(employee_id)
```



design pattern is called the [singleton](#) design pattern, which comes in handy for classes from which there should only be one single instance.

Now, you can work on the `employees` module. You'll also mark the `EmployeeDatabase` as internal and make a singleton out of it, but you'll make some additional changes:

```
Python                                     employees.py

from productivity import get_role
from hr import get_policy
from contacts import get_employee_address
from representations import AsDictionaryMixin

class _EmployeeDatabase:
    def __init__(self):
        self._employees = {
            1: {"name": "Mary Poppins", "role": "manager"},
            2: {"name": "John Smith", "role": "secretary"},
            3: {"name": "Kevin Bacon", "role": "sales"},
            4: {"name": "Jane Doe", "role": "factory"},
            5: {"name": "Robin Williams", "role": "secreta
        }

    @property
    def employees(self):
        return [Employee(id_) for id_ in sorted(self._empl

    def get_employee_info(self, employee_id):
        info = self._employees.get(employee_id)
        if not info:
            raise ValueError(employee_id)
        return info

class Employee(AsDictionaryMixin):
    def __init__(self, id):
        self.id = id
        info = employee_database.get_employee_info(self.id
        self.name = info.get("name")
        self.address = get_employee_address(self.id)
        self._role = get_role(info.get("role"))
        self._payroll = get_policy(self.id)
```



```

print(f"- {duties}")
print("")
self._payroll.track_work(hours)

def calculate_payroll(self):
    return self._payroll.calculate_payroll()

employee_database = _EmployeeDatabase()

```

You first import the relevant public functions and classes from other modules. You make `_EmployeeDatabase` internal, and at the bottom, you create a single instance. This instance is public and part of the interface because you'll want to use it in the application.

You changed the `_EmployeeDatabase._employees` attribute to a dictionary where the key is the employee ID and the value is the employee information. You also exposed a `.get_employee_info()` method to return the information for the specified employee `employee_id`.

The `_EmployeeDatabase.employees` property now sorts the keys to return the employees sorted by their `.id`. You replaced the method that constructed the `Employee` objects with calls to the `Employee` initializer directly.

The `Employee` class now is initialized with the ID and uses the public functions exposed in the other modules to initialize its attributes.

You can now change the program to test the changes:

```

Python                                program.py

import json

from hr import calculate_payroll
from productivity import track
from employees import employee_database, Employee

def print_dict(d):
    print(json.dumps(d, indent=2))

```



```
calculate_payroll(employees)

temp_secretary = Employee(5)
print("Temporary Secretary:")
print_dict(temp_secretary.to_dict())
```

You import the relevant functions from the `hr` and `productivity` modules, as well as the `employee_database` and `Employee` class. The program is cleaner because you exposed the required interface and encapsulated how to access objects.

Notice that you can now create an `Employee` object directly just using its ID. You can run the program to see its output:

Shell



```
$ python program.py
```

```
Tracking Employee Productivity
=====
Employee 1 - Mary Poppins:
- screams and yells for 40 hours.

Employee 2 - John Smith:
- does paperwork for 40 hours.

Employee 3 - Kevin Bacon:
- expends 40 hours on the phone.

Employee 4 - Jane Doe:
- manufactures gadgets for 40 hours.

Employee 5 - Robin Williams:
- does paperwork for 40 hours.

Calculating Payroll
=====
Payroll for: 1 - Mary Poppins
- Check amount: 3000
- Sent to:
121 Admin Rd.
Concord, NH 03301
```



- Sent to:
67 Paperwork Ave
Manchester, NH 03101

Payroll for: 3 - Kevin Bacon
- Check amount: 1800.0
- Sent to:
15 Rose St
Apt. B-1
Concord, NH 03301

Payroll for: 4 - Jane Doe
- Check amount: 600
- Sent to:
39 Sole St.
Concord, NH 03301

Payroll for: 5 - Robin Williams
- Check amount: 360
- Sent to:
99 Mountain Rd.
Concord, NH 03301

```
Temporary Secretary:  
{  
    "id": "5",  
    "name": "Robin Williams",  
    "address": {  
        "street": "99 Mountain Rd.",  
        "street2": "",  
        "city": "Concord",  
        "state": "NH",  
        "zipcode": "03301"  
    }  
}
```

The program works the same as before, but now you can see that you can create a single `Employee` object from its ID and display its dictionary representation.

Take a closer look at the `Employee` class:

Python

`employees.py`



```
class Employee(ASDictionaryMixin):
    def __init__(self, id):
        self.id = id
        info = employee_database.get_employee_info(self.id)
        self.name = info.get("name")
        self.address = get_employee_address(self.id)
        self._role = get_role(info.get("role"))
        self._payroll = get_policy(self.id)

    def work(self, hours):
        duties = self._role.perform_duties(hours)
        print(f"Employee {self.id} - {self.name}:")
        print(f"- {duties}")
        print("")
        self._payroll.track_work(hours)

    def calculate_payroll(self):
        return self._payroll.calculate_payroll()
```

The `Employee` class is a composite that contains multiple objects providing different functionality. It contains an `Address` that implements all the functionality related to where the employee lives.

`Employee` also contains a productivity role from the `productivity` module, and a payroll policy from the `hr` module. These two objects provide implementations that the `Employee` class leverages to track work in the `.work()` method and to calculate the payroll in the `.calculate_payroll()` method.

You're using composition in two different ways. The `Address` class provides additional data to `Employee`, while the `role` and `payroll` objects provide additional behavior.

Still, the relationship between `Employee` and those objects is loosely coupled, which provides some interesting capabilities that you'll see in the next section.

Composition to Change Run-Time Behavior



customize behavior. Method overriding is the only way to customize the behavior of a base class. This creates rigid designs that are difficult to change.

Composition, on the other hand, provides a loosely coupled relationship that enables flexible designs and can be used to change behavior at runtime.

Imagine you need to support a long-term disability (LTD) policy when calculating payroll. The policy states that an employee on LTD should be paid 60 percent of their weekly salary, assuming forty hours of work.

With an inheritance design, this can be a very difficult requirement to support. Adding it to the composition example is a lot simpler. Start by adding the policy class:

```
Python                                                                    hr.py

# ...

class LTDPolicy:
    def __init__(self):
        self._base_policy = None

    def track_work(self, hours):
        self._check_base_policy()
        return self._base_policy.track_work(hours)

    def calculate_payroll(self):
        self._check_base_policy()
        base_salary = self._base_policy.calculate_payroll()
        return base_salary * 0.6

    def apply_to_policy(self, base_policy):
        self._base_policy = base_policy

    def _check_base_policy(self):
        if not self._base_policy:
            raise RuntimeError("Base policy missing")

# ...
```



is completely different, so you don't want to inherit any of the `PayrollPolicy` implementation.

The `LTDPolicy` initializes `._base_policy` to `None` and provides an internal `._check_base_policy()` method that raises an exception if the `._base_policy` hasn't been applied. Then, it provides an `._apply_to_policy()` method to assign `._base_policy`.

The public interface first checks that `._base_policy` has been applied, and then it implements the functionality in terms of that base policy. The `._track_work()` method just delegates to the base policy, and `._calculate_payroll()` uses it to calculate the `base_salary` and then return the 60 percent.

You can now make a small change to the `Employee` class:

```
Python                                     employees.py

# ...

class Employee(AsDictionaryMixin):

    # ...

    def apply_payroll_policy(self, new_policy):
        new_policy.apply_to_policy(self._payroll)
        self._payroll = new_policy
```

You added an `._apply_payroll_policy()` method that applies the existing payroll policy to the new policy and then substitutes it. You can now modify the program to apply the policy to an `Employee` object:

```
Python                                     program.py

from hr import calculate_payroll, LTDPolicy
from productivity import track
from employees import employee_database

employees = employee_database.employees

sales_employee = employees[2]
```



```
track(employees, 40)
calculate_payroll(employees)
```

The program accesses `sales_employee` located at index 2, creates the `LTDPolicy` object, and applies the policy to the employee. When you call `.calculate_payroll()`, the change is reflected. You can run the program to evaluate the output:

Shell



```
$ python program.py
```

```
Tracking Employee Productivity
```

```
=====
```

```
Employee 1 - Mary Poppins:
```

```
- screams and yells for 40 hours.
```

```
Employee 2 - John Smith:
```

```
- does paperwork for 40 hours.
```

```
Employee 3 - Kevin Bacon:
```

```
- expends 40 hours on the phone.
```

```
Employee 4 - Jane Doe:
```

```
- manufactures gadgets for 40 hours.
```

```
Employee 5 - Robin Williams:
```

```
- does paperwork for 40 hours.
```

```
Calculating Payroll
```

```
=====
```

```
Payroll for: 1 - Mary Poppins
```

```
- Check amount: 3000
```

```
- Sent to:
```

```
121 Admin Rd.
```

```
Concord, NH 03301
```

```
Payroll for: 2 - John Smith
```

```
- Check amount: 1500
```

```
- Sent to:
```

```
67 Paperwork Ave
```

```
Manchester, NH 03101
```



15 Rose St
Apt. B-1
Concord, NH 03301

Payroll for: 4 - Jane Doe
- Check amount: 600
- Sent to:
39 Sole St.
Concord, NH 03301

Payroll for: 5 - Robin Williams
- Check amount: 360
- Sent to:
99 Mountain Rd.
Concord, NH 03301

The check amount for employee Kevin Bacon, who's the sales employee, is now for 1080 dollars instead of 1800 dollars. That's because the `LTDPolicy` has been applied to the salary.

As you can see, you were able to support the changes just by adding a new policy and modifying a couple of interfaces. This is the kind of flexibility that policy design based on composition gives you.

Choosing Between Inheritance and Composition in Python

Python, as an object-oriented programming language, supports both inheritance and composition. You saw that inheritance is best used to model an **is a** relationship, whereas composition models a **has a** relationship.

Sometimes, it's hard to see what the relationship between two classes should be, but you can follow these guidelines:

- **Use inheritance over composition in Python** to model a clear **is a** relationship. First, justify the relationship between the derived class and its base. Then, reverse the relationship and try to justify it. Only if you *can't* justify the relationship in both directions should you use inheritance between them.



- **Use inheritance over composition in Python** to provide **mixins** features to several unrelated classes when there's only one implementation of that feature.
- **Use composition over inheritance in Python** to model a **has a** relationship that leverages the implementation of the component class.
- **Use composition over inheritance in Python** to create components that multiple classes in your Python applications can reuse.
- **Use composition over inheritance in Python** to implement groups of behaviors and policies that can be applied interchangeably to other classes to customize their behavior.
- **Use composition over inheritance in Python** to enable runtime behavior changes without affecting existing classes.

With that, you have a strong understanding of when to use inheritance vs composition.

Conclusion

You explored **inheritance and composition in Python**. You learned about the type of relationships that inheritance and composition create. You also went through a series of exercises to understand how inheritance and composition are implemented in Python.

In this tutorial, you learned how to:

- Use **inheritance** to express an **is a** relationship between two classes
- Evaluate if inheritance is the **right relationship**
- Use multiple inheritance in Python and evaluate Python's MRO to **troubleshoot multiple inheritance** problems
- Extend classes with **mixins** and reuse their implementation



- Provide **flexible designs** using composition
- Reuse existing code through **policy design** based on composition

Inheritance and composition are both important concepts in [object-oriented programming](#). By working through this tutorial, you've strengthened your understanding of how you can use them in your Python programs and when you should choose one over the other.

Recommended Reading

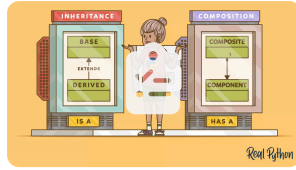
Below, you'll find a list of books and articles that further explore object-oriented design. These can help you understand the correct use of inheritance and composition in Python and other programming languages:

- [Design Patterns: Elements of Reusable Object-Oriented Software](#)
- [Head First Design Patterns: A Brain-Friendly Guide](#)
- [Clean Code: A Handbook of Agile Software Craftsmanship](#)
- [SOLID Principles: Improve Object-Oriented Design in Python](#)
- [Liskov Substitution Principle](#)

What's your main takeaway from reading about inheritance and composition in Python? Share your thoughts in the comments below.

Get Your Code: [Click here to get the free sample code](#) that shows you how to use inheritance and composition in Python.

 **Take the Quiz:** Test your knowledge with our interactive "Inheritance and Composition: A Python OOP Guide" quiz. You'll receive a score upon completion to help you track your learning progress:



Python OOP Guide

In this quiz, you'll test your understanding of inheritance and composition in Python. These are two major concepts in object-oriented programming that help model the relationship between two classes. By working through this quiz, you'll revisit how to use inheritance and composition in Python, model class hierarchies, and use multiple inheritance.

Frequently Asked Questions

Now that you have some experience with inheritance and composition in Python, you can use the questions and answers below to check your understanding and recap what you've learned.

These FAQs are related to the most important concepts you've covered in this tutorial. Click the *Show/Hide* toggle beside each question to reveal the answer.

What are composition and inheritance in Python?

Show/Hide

How is composition achieved in Python?

Show/Hide

What is the difference between composition and inheritance?

Show/Hide

How to achieve inheritance in Python?

Show/Hide

Mark as Completed



Share



Python Tricks

Get a short & sweet **Python Trick** delivered to your inbox every couple of days. No spam ever. Unsubscribe any time. Curated by the Real Python team.

```
1 # How to merge two dicts
2 # in Python 3.5+
3
4 >>> x = {'a': 1, 'b': 2}
5 >>> y = {'b': 3, 'c': 4}
6
7 >>> z = {**x, **y}
8
9 >>> z
10 {'c': 4, 'a': 1, 'b': 3}
```

Send Me Python Tricks »

About Isaac Rodriguez



Hi, I'm Isaac. I build, lead, and mentor software development teams, and for the past few years I've been focusing on cloud services and back-end applications using Python among other languages. Love to hear from you here at Real Python.

» [More about Isaac](#)

Each tutorial at Real Python is created by a team of developers so that it meets our high quality standards. The team members who worked on this tutorial are:



Geir Arne



Joanna



Kate

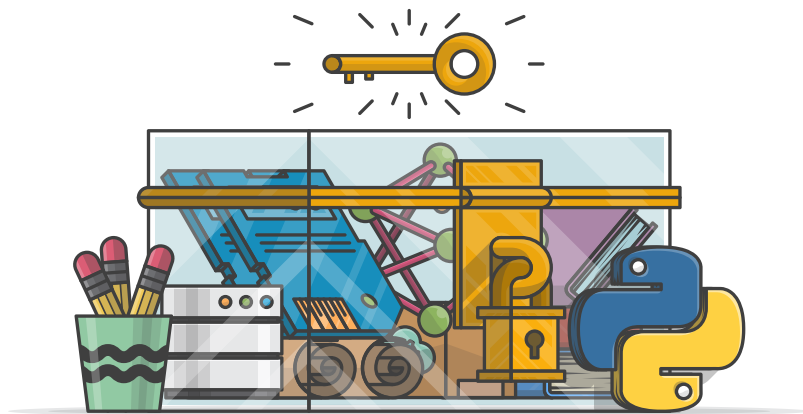


Martin



Stephen

Master Real-World Python Skills With Unlimited Access to Real Python



Join us and get access to thousands of tutorials, hands-on video courses, and a community of expert Pythonistas:

[Level Up Your Python Skills »](#)

What Do You Think?

Rate this article:



[LinkedIn](#)

[Twitter](#)

[Bluesky](#)

[Facebook](#)

[Email](#)

What's your #1 takeaway or favorite thing you learned? How are you going to put your newfound skills to use? Leave a comment



Commenting Tips: The most useful comments are those written with the goal of learning from or helping out other students. Get tips for asking good questions and get answers to common questions in our support portal.

Looking for a real-time conversation? Visit the Real Python Community Chat or join the next “Office Hours” Live Q&A Session. Happy Pythoning!

Keep Learning

Related Topics: [intermediate](#) [best-practices](#) [python](#)

Related Learning Paths:

- [Object-Oriented Programming \(OOP\)](#)

Related Courses:

- [Inheritance and Composition: A Python OOP Guide](#)

Related Tutorials:

- [Supercharge Your Classes With Python super\(\)](#)
- [Python Classes: The Power of Object-Oriented Programming](#)
- [Object-Oriented Programming \(OOP\) in Python](#)
- [Python's Magic Methods: Leverage Their Power in Your Classes](#)
- [The Factory Method Pattern and Its Implementation in Python](#)

Learn Python

[Start Here](#)

Courses & Paths

[Learning Paths](#)

Community

[Podcast](#)

[Python Reference](#)[Python Cheat Sheet](#)[Support Center](#)[Live Courses](#)[Books](#)[Office Hours](#)[Learner Stories](#)

Membership

[Plans & Pricing](#)[Team Plans](#)[For Business](#)[For Schools](#)[Reviews](#)

Company

[About Us](#)[Team](#)[Mission & Values](#)[Editorial Guidelines](#)[Sponsorships](#)[Careers](#)[Press Kit](#)[Merch](#)[Privacy Policy](#) · [Terms of Use](#) · [Security](#) · [Contact](#)

❤ Happy Pythoning!

© 2012–2026 DevCademy Media Inc. DBA Real Python. All rights reserved.
REALPYTHON™ is a trademark of DevCademy Media Inc.

